

Traffic-MoE: A Sparse Foundation Model for Network Traffic Analysis

Jiajun Zhou
Zhejiang University of Technology
Hangzhou, China
jjzhou@zjut.edu.cn

Changhui Sun
Zhejiang University of Technology
Hangzhou, China
chenjinsong@hust.edu.cn

Meng Shen
Beijing Institute of Technology
Beijing, China
shenmeng@bit.edu.cn

Shanqing Yu
Zhejiang University of Technology
Hangzhou, China
yushanqing@zjut.edu.cn

Qi Xuan
Zhejiang University of Technology
Hangzhou, China
xuanqi@zjut.edu.cn

Abstract

While pre-trained large models have achieved state-of-the-art performance in network traffic analysis, their prohibitive computational costs hinder deployment in real-time, throughput-sensitive network defense environments. This work bridges the gap between advanced representation learning and practical network protection by introducing *Traffic-MoE*, a sparse foundation model optimized for high-efficiency real-time inference. By dynamically routing traffic tokens to a small subset of specialized experts, *Traffic-MoE* effectively decouples model capacity from computational overhead. Extensive evaluations across three security-oriented tasks demonstrate that *Traffic-MoE* achieves up to a 12.38% improvement in detection performance compared to leading dense competitors. Crucially, it delivers a 91.62% increase in throughput, reduces inference latency by 47.81%, and cuts peak GPU memory consumption by 38.72%. Beyond efficiency, *Traffic-MoE* exhibits superior robustness against adversarial traffic shaping and maintains high detection efficacy in few-shot scenarios, establishing a new paradigm for scalable and resilient network traffic analysis.

CCS Concepts

• Security and privacy → Network security.

Keywords

Network Traffic Analysis, Mixture-of-Experts, Pretraining

ACM Reference Format:

Jiajun Zhou, Changhui Sun, Meng Shen, Shanqing Yu, and Qi Xuan. 2025. Traffic-MoE: A Sparse Foundation Model for Network Traffic Analysis. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (CCS'25)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS'25, Taipei, ON, Taiwan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Network communication has rapidly evolved with the widespread adoption of encryption, heterogeneous device ecosystems, and emerging paradigms such as IoT and Web3 [1, 21]. Today, TLS-based services [23], device-to-cloud telemetry, and decentralized protocols dominate Internet traffic, obscuring application semantics from traditional packet inspection [34]. While these advancements improve privacy and availability, they also pose a critical security challenge: malicious activities, such as botnet communications and credential misuse, can hide within encrypted traffic and remain indistinguishable from benign services at the packet level [2, 42].

Traffic analysis is fundamental to network defense, supporting tasks such as intrusion detection [52], anomaly detection [11], service attribution [38], and access control. However, classical machine-learning approaches rely on handcrafted statistical features or protocol signatures, which are fragile under encryption, obfuscation, and protocol mimicry. To address these limitations, recent work leverages deep learning models that operate directly on raw byte sequences or image-based representations, achieving higher accuracy on heterogeneous and encrypted traffic [5, 27, 29]. Following trends in computer vision (CV) and natural language processing (NLP) [14], state-of-the-art (SOTA) pre-trained traffic analysis models increasingly leverage large transformer-based architectures to further improve downstream performance [16, 26, 30, 57].

Despite these advances, most models employ *dense* architectures that activate all parameters during inference, causing computational cost to scale with model size. As model capacity grows, inference latency becomes prohibitively high, hindering deployment in real-time environments such as edge gateways, IoT devices, and high-throughput intrusion detection systems. Although pre-trained models capture rich protocol semantics, their dense architectures force a trade-off between accuracy and deployability: large models may perform well on offline benchmarks yet struggle in operational environments where high throughput and low latency are critical. A practical defense system therefore requires models that preserve semantic expressiveness while enabling efficient inference.

We address this deployment gap by rethinking how large-scale traffic analysis models should allocate computation. We observe that network traffic behaviors are structurally heterogeneous rather than uniformly distributed, as malware beacons, VPN tunnels, Web3 relay traffic, and benign application flows all exhibit distinct byte-level patterns and temporal dynamics [17, 42]. A uniform dense

architecture allocates equal computational budget to all patterns, whereas a specialized-expert model can assign computation proportionally to the traffic domain being processed [24, 41]. This leads to our central insight: *Efficiency in network traffic modeling does not necessarily stem from shrinking models, but rather from selectively activating model capacity during inference* [10, 20].

In this work, we propose *Traffic-MoE*, a sparse Mixture-of-Experts (MoE) [41] foundation model for network traffic analysis. Instead of activating all parameters during inference, *Traffic-MoE* routes each token through a small subset of specialized experts, decoupling total model capacity from computational overhead. This design enables scaling the model parameter count by orders of magnitude without proportionally increasing inference cost, making large-scale pre-trained traffic analysis models feasible for latency-sensitive scenarios. To support heterogeneous network flows, we further develop *Traffic2Token*, a unified byte-level representation method combining protocol metadata with selective payload segments, enabling pre-training without full payload visibility. *Traffic-MoE* is trained using autoregressive self-supervision over massive unlabeled traffic corpora, capturing protocol semantics, flow directionality, and temporal dependencies. During downstream adaptation, *Traffic-MoE* can be fine-tuned for intrusion detection, service classification, VPN/Tor analysis, and other security tasks while preserving real-time inference performance, making it suitable not only for benchmarking but for operational deployment. The main contributions of this work are summarized as follows:

- **A sparse foundation model for network traffic analysis.** We propose *Traffic-MoE*, the first MoE architecture tailored to traffic modeling, enabling large model capacity while maintaining real-time inference, bridging the efficiency gap that limits current dense pre-trained traffic analysis models.
- **A unified learning framework enabling expert specialization across traffic semantics.** We support heterogeneous network flows through *Traffic2Token* and balanced sparse routing, allowing experts to autonomously specialize in distinct traffic behaviors and improving security-oriented generalization.
- **Extensive evaluations demonstrate the superiority and efficiency.** Compared to SOTA dense competitors, *Traffic-MoE* achieves up to a 12.38% improvement in detection performance, a 91.62% increase in throughput, and reductions of 47.81% in inference latency and 38.72% in GPU memory consumption. Furthermore, it maintains superior performance under few-shot scenarios and exhibits robustness against distribution shifts.

2 Related Work

2.1 Statistical Machine Learning Methods

Early traffic analysis techniques rely on Deep Packet Inspection (DPI), which identifies applications by matching predefined signatures within packet headers or payloads [49]. While effective in unencrypted scenarios, DPI incurs high computational overhead and becomes unreliable as modern protocols increasingly employ end-to-end encryption and payload obfuscation. To overcome reliance on payload visibility, statistical feature-based machine learning approaches extract handcrafted temporal or structural features for classification. Moore et al. [31] reduce dependence on port-based heuristics by focusing on payload features, while Saber et

al. [39] combine PCA with SVM to model time-based statistical patterns. AppScanner [46] extracts 54 statistical features from encrypted traffic and trains a Random Forest classifier to identify mobile applications. FlowPrint [47] leverages clustering to capture temporal correlations among network flows and generates application-specific traffic fingerprints, enabling the identification of previously unseen applications. Although computationally efficient, these methods depend heavily on feature engineering and struggle to capture deep semantic patterns, leading to degraded performance under encrypted or adversarial traffic conditions.

2.2 Deep Learning Methods

Following the rise of deep learning (DL), end-to-end models have been widely adopted to automatically extract features from raw traffic data, reducing reliance on handcrafted features. These approaches handle traffic using various data representations and model architectures. For sequence modeling, FS-Net [27] employs a bidirectional GRU-based encoder-decoder architecture to directly learn features from encrypted traffic flows. For image-based representations, ATVITSC [28] transforms payload bytes into images and combines vision transformers with LSTM modules to jointly capture spatial and temporal dependencies. Graph-based models further treat flows as structured relational data. TFE-GNN [54] encodes packet headers and payloads as graph nodes, while Zhou et al. [58] introduce hypergraph structure that capture higher-order correlations through KNN-derived hyperedges. These methods demonstrate the ability of deep neural networks to learn semantic structure beyond handcrafted features. However, they remain heavily dependent on large-scale labeled datasets, which are expensive to construct in real-world environments [43]. Label scarcity is especially pronounced in critical long-tail classes such as zero-day attacks and malware variants, where samples are sparse and annotation requires expert knowledge [15]. As a result, even advanced DL approaches experience significant degraded performance when deployed in practical scenarios with limited labeled data.

2.3 Large Pre-trained Methods

Recent studies introduce pre-training paradigm into traffic analysis. Models such as PERT [16], ET-BERT [26], NetGPT [30], TrafficFormer [57] and YaTC [56] perform self-supervised learning on large-scale traffic corpora to acquire generalizable representations before fine-tuning. These methods significantly improve performance on network traffic detection under limited supervision, demonstrating the promise of the pre-train and fine-tune paradigm. However, existing models generally adopt dense transformer-based architectures, where all parameters are activated during inference. As noted in [41], model capacity and inference cost remain tightly coupled, leading to high latency and prohibitive resource consumption that limit deployment in edge, IoT, and online scenarios.

3 Threat Model

Deployment Scenario. A hierarchical monitoring architecture is considered for enterprise networks or ISPs [44, 55] in Figure 1. To balance throughput with inspection depth, a two-stage filtering mechanism is deployed at the network edge. A primary lightweight filter first handles distinct plaintext threats [37, 44]. Subsequently,

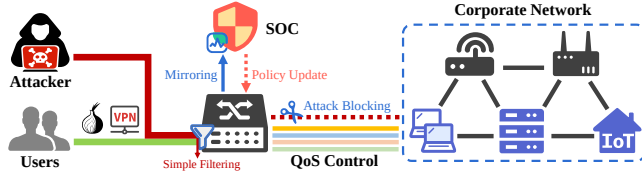


Figure 1: Threat model and operational scenario.

specialized deep inspection is reserved for ambiguous, high-risk, or encrypted flows mirrored to the analysis engine via out-of-path inspection. This design ensures zero impact on network latency while allowing detection results to trigger dynamic policy updates (e.g., blocking or QoS adjustment) in real-time [7, 19]. Crucially, given the immense volume of backbone traffic, the inference engine requires high parameter efficiency to maintain real-time processing capabilities without imposing prohibitive computational overheads.

Attacker Capabilities. We assume a sophisticated adversary aiming to disrupt services or exfiltrate data while evading detection [4]. The adversary employs encryption and anonymization to conceal payload contents and communication endpoints [9, 51]. To defeat statistical analysis, traffic shaping techniques are utilized, including manipulating packet inter-arrival times and dynamically altering flow composition ratios to distort flow fingerprints [3, 59]. Furthermore, these distribution shift strategies are actively employed to generate out-of-distribution (OOD) traffic or zero-day attacks [25], attempting to bypass models trained on stationary datasets.

Knowledge and Constraints. Based on a black-box assumption, adversary knows the defense architecture but lacks white-box access to model parameters or gradients [35]. Additionally, attack strategies are bound by functional constraints, and traffic manipulation must not compromise the reliability of the command-and-control channel or render communication indistinguishable from random noise, as perfect mimicry typically incurs unacceptable operational costs [18, 40, 43].

Defender Capabilities. While the defender possesses full visibility of mirrored traffic headers and encrypted payloads, operations are conducted under strict constraints. Specifically, due to privacy regulations and protocols implementing Perfect Forward Secrecy (PFS), payloads cannot be decrypted [36], compelling the inference engine to rely exclusively on plaintext metadata, temporal sequences, and underlying behavioral patterns [6, 12]. Furthermore, we operate under the assumption of a Trusted Infrastructure where the traffic collection and analysis pipeline is secure. Consequently, threats such as physical compromise of the SOC or poisoning of the pre-training data are considered out of scope.

4 Traffic-MoE Architecture

Network traffic exhibits heterogeneous behavioral patterns that necessitate domain-specific modeling rather than uniform dense computation. Meanwhile, real-world security monitoring systems require low-latency inference, making dense large-scale models impractical. These challenges create a deployment gap: semantic modeling of network traffic is increasingly effective but not computationally feasible in operational environments. To address these constraints, *Traffic-MoE* is designed as a sparse foundation model

for network traffic analysis that balances semantic expressiveness with computational efficiency. Instead of shrinking model capacity, *Traffic-MoE* activates only a subset of model parameters per input, enabling scalability without proportional inference overhead. This sparse activation strategy adapts the computational pathway to specific traffic semantics, enabling domain specialization across heterogeneous flows. Figure 2 presents the overall workflow. Specifically, *Traffic-MoE* consists of four components: 1) Traffic2Token converts raw flows into structured byte-level token sequences that preserve critical metadata and structural patterns; 2) A hierarchical transformer backbone models temporal dependencies across packets to obtain contextualized embeddings; 3) Sparse MoE layers replace dense feed-forward layers and activate only top- k experts based on token semantics, combining shared experts with domain-specialized ones; 4) A pretrain-finetune pipeline enables large-scale self-supervised learning and downstream task adaptation, supporting scenarios such as service classification and malicious traffic detection. Together, these components form a unified architecture that preserves semantic modeling capability while ensuring high efficiency in throughput-sensitive environments.

4.1 Traffic Tokenization

Raw traffic collected from operational networks contains heterogeneous communication patterns across applications, protocols, and devices. Directly modeling raw packets is ineffective as full headers introduce substantial noise while encrypted payloads limit semantic visibility. So we design a Traffic2Token module that converts raw traffic into structured token sequences, preserving byte-level behavioral signals while avoiding protocol-specific assumptions.

4.1.1 Packet-Based Flow Serialization. We first reconstruct session-level flows from packet traces using five-tuple identifiers (source/destination IPs, ports, and transport protocol). Flows that contain too few packets are removed to reduce statistical variance and ensure reliable behavioral patterns. The resulting session flows form the basic processing unit of Traffic2Token. To avoid introducing noise from randomized or transport-irrelevant header fields, each packet is transformed into a compact and stable representation. Specifically, we extract six packet-level attributes that reflect transport behavior and timing dynamics: packet length, transmission direction, TCP flag bits, inter-arrival time, IP protocol type, and payload length, as shown in Figure 3. These attributes are serialized into a fixed-length byte sequence. In parallel, we sample a short segment of J payload bytes to retain representative content patterns while controlling sequence length. The metadata bytes and sampled payload bytes are then concatenated to form the packet-level hex sequence. For each flow, we select the first K packets as model input. The hex representations of these packets are concatenated in chronological order to construct a flow-level hex sequence. This packet-based serialization preserves communication directionality, burstiness, and timing structure, which are often lost in purely byte-level streams.

4.1.2 Tokenization and Embedding. Network protocols often exhibit short-range structural regularities such as protocol-specific header fields, framing patterns, and data boundaries. To expose such correlations, we apply a sliding window to the flow hex sequence and convert adjacent byte pairs into byte bigrams. This bigram

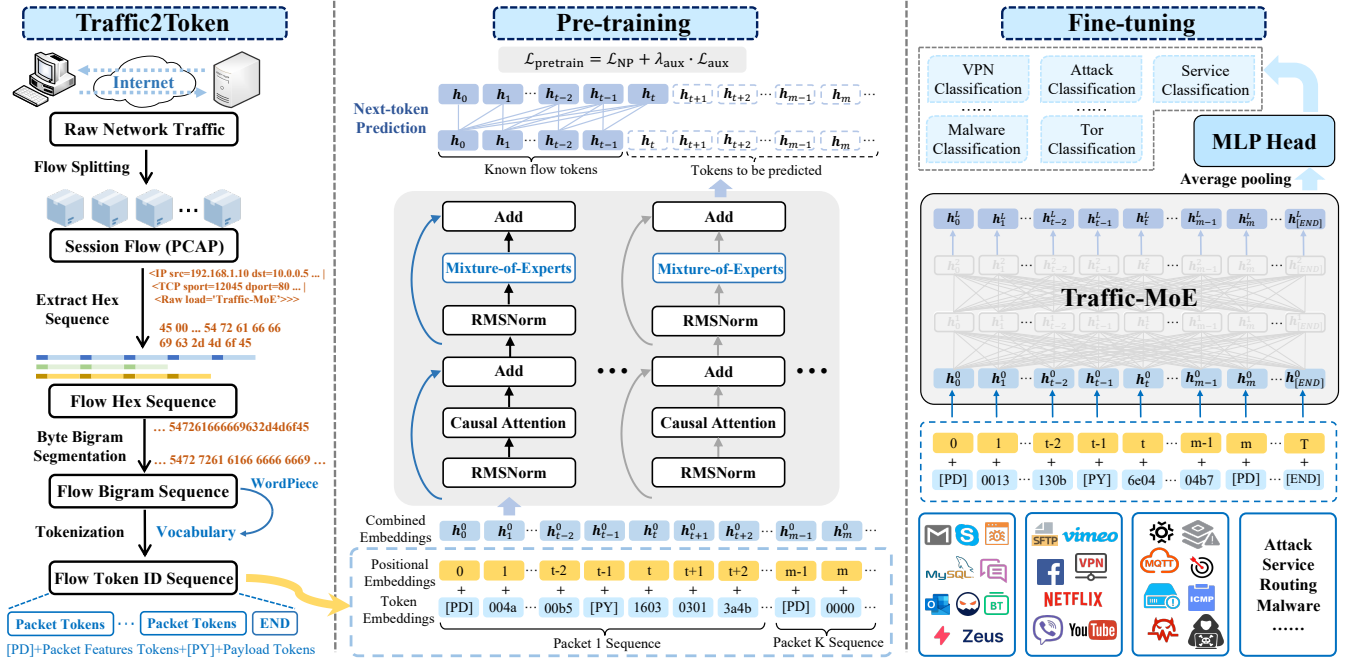


Figure 2: The overall framework of *Traffic-MoE*. It consists of the Traffic2Token serialization module, the sparse Mixture-of-Experts (MoE) backbone, and the pre-training/fine-tuning pipeline for downstream security tasks.

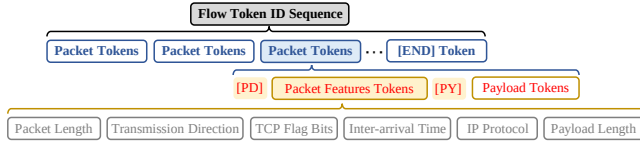


Figure 3: Illustration of flow token sequence representation.

transformation yields a fine-grained sequence that highlights local structural cues without relying on full header retention. We then treat the bigram sequences from all flows as a corpus and employ the WordPiece [50] algorithm to construct a sub-byte-level vocabulary $\mathcal{V}$. WordPiece adaptively identifies statistically significant bigram combinations (e.g., TLS record headers) and assigns them as token units. This data-driven vocabulary captures recurrent traffic patterns while preventing the vocabulary from exploding. Furthermore, to enhance the model’s perception of the session flow’s hierarchical structure, we introduce function-specific markers into the sequence: [PD] marks the beginning of each packet sequence; [PY] separates metadata fields from payload bytes; [PAD] is used for sequence padding to ensure consistent input length; [END] marks the termination of the flow sequence; and [UNK] represents low-frequency byte patterns outside the vocabulary. These structural markers ensure that packet boundaries remain clearly identifiable to the model when processing long sequences, preventing the semantic collapse of flow information within a flattened byte stream. Ultimately, the bigram tokens and special markers are finally mapped to Token IDs via the learned vocabulary. Each Token ID is embedded into

a dense vector and summed with a positional encoding, producing the flow token sequence representation: $X = \{x_1, x_2, \dots, x_T\}$, where $x_t \in \mathbb{R}^d$ is the initial token embedding in position t , T is the sequence length, and d is the embedding dimension. This unified embedding supports both local byte-level pattern modeling and global packet-level temporal reasoning, enabling robust pre-training and sparse expert routing. Compared to traditional feature engineering or header-based encoding, Traffic2Token preserves deep structural behavior of network traffic, thereby supporting heterogeneous downstream tasks such as VPN/Tor identification, malicious traffic detection, and service classification.

4.2 Backbone with Sparse Expert Layers

Figure 4 illustrates the overall backbone design of *Traffic-MoE*, which integrates causal masked self-attention with a sparse MoE feed-forward layer. This architecture is motivated by the dual nature of network traffic: packet sequences exhibit strong temporal dependencies driven by protocol execution (e.g., handshakes, tunneling transitions), while simultaneously containing heterogeneous and domain-specific behaviors produced by diverse applications and threat actors. To capture these sequential structures and behavioral variations within a unified model, the backbone adopts an autoregressive transformer stack in which each block first performs causal self-attention to model the ordered evolution of flow tokens and then applies a sparsely activated expert layer to adapt computation to the underlying traffic semantics. The resulting hybrid design decouples representational capacity from inference cost, enabling *Traffic-MoE* to scale to large parameter counts while remaining efficient for real-time inference.

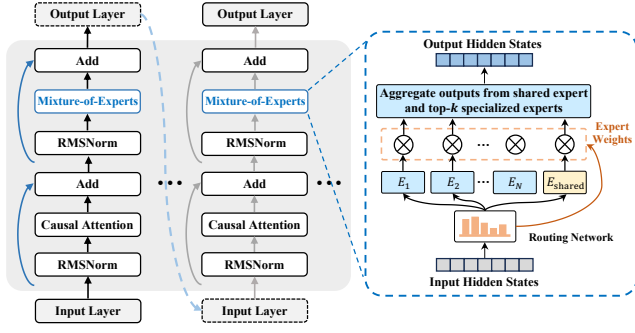


Figure 4: Illustration of *Traffic-MoE* backbone architecture.

4.2.1 Causal Masked Self-Attention. Network flows evolve through ordered protocol phases such as handshake exchanges, tunneling encapsulation, retransmissions, and periodic keep-alives, which manifest as temporal structures in their byte-level token sequences. To model these sequential dependencies while respecting real-world monitoring constraints where future packets are unavailable at inference time, *Traffic-MoE* employs a causal masked multi-head self-attention mechanism.

Given the hidden states $H^{(l-1)} \in \mathbb{R}^{T \times d}$ from the $(l-1)$ -th block ($H^0 = X$), we first apply Root Mean Square Layer Normalization (RMSNorm) [53] to stabilize the feature scale across heterogeneous flow segments and improve training stability. Subsequently, to encode temporal offsets between packets, such as burst intervals or phase transitions, we incorporate Rotary Positional Embeddings (RoPE) [45] into the query and key projections. Unlike absolute positional encodings, RoPE mathematically injects relative position information by rotating the feature vectors in a shared frequency space. The query (Q), key (K), and value (V) matrices for the j -th attention head are computed as:

$$\begin{aligned} Q_j^{(l)} &= \text{RoPE} \left(\text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{Q,j}^{(l)} \right), \\ K_j^{(l)} &= \text{RoPE} \left(\text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{K,j}^{(l)} \right), \\ V_j^{(l)} &= \text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{V,j}^{(l)}, \end{aligned} \quad (1)$$

where $W_{\{Q,K,V\},j}^{(l)} \in \mathbb{R}^{d \times d_m}$ are learnable projection matrices. This formulation allows the attention mechanism to naturally capture the relative sequential distance between flow tokens, which is critical for identifying traffic patterns invariant to absolute sequence positions (e.g., attack signatures appearing at varying offsets).

To adhere to the physical constraint of network traffic where future packets cannot influence current state, we apply a strictly lower-triangular causal mask $M \in \mathbb{R}^{T \times T}$. The attention output is:

$$O_j^{(l)} = \text{Softmax} \left(\frac{Q_j^{(l)} (K_j^{(l)})^\top}{\sqrt{d_m}} + M \right) \cdot V_j^{(l)}, \quad M_{tp} = \begin{cases} 0, & p \leq t \\ -\infty, & p > t \end{cases} \quad (2)$$

This causal mask ensures that the token at position t can only attend to information from itself and preceding positions. Finally, the outputs from all m attention heads are concatenated and projected

to form the residual update:

$$\tilde{H}^{(l)} = H^{(l-1)} + \left[O_1^{(l)} \parallel O_2^{(l)} \parallel \dots \parallel O_m^{(l)} \right] \cdot W_O^{(l)}, \quad (3)$$

where $W_O^{(l)} \in \mathbb{R}^{d \times d}$ is the output projection matrix, and $\tilde{H}^{(l)}$ is the intermediate hidden state. This design ensures that the model learns underlying traffic semantics autoregressively, effectively modeling the state transition logic of network protocols.

4.2.2 Mixture-of-Experts Layer with Sparse Routing. Following the causal masked self-attention module, the intermediate hidden state $\tilde{H}^{(l)}$ is processed by a sparse MoE layer. Standard Transformers typically employ a dense Feed-Forward Network (FFN) that applies identical parameters to all tokens. However, network traffic combines universal protocol regularities (e.g., transport headers, handshake patterns) with highly heterogeneous behaviors (e.g., application-specific traffic fingerprints, malware command-and-control flows). To accommodate this heterogeneity without incurring a proportional increase in computational cost, *Traffic-MoE* replaces the dense FFN with a hybrid MoE layer composed of a shared expert and multiple specialized experts.

To decouple general protocol knowledge from specific traffic behaviors, we design a dual-pathway computation. First, the input $\tilde{H}^{(l)}$ undergoes normalization via RMSNorm to produce $Z^{(l)} = \text{RMSNorm}(\tilde{H}^{(l)})$. This normalized representation is then routed to two distinct expert groups, as defined below:

1) Shared Expert for Universal Protocol Semantics. To capture invariant traffic structures distinct from application-specific payloads, we employ a persistently activated *shared expert*, E_{shared} . Unlike the competitive routing of specialized experts, the shared expert utilizes an independent gating mechanism to adaptively regulate the injection of general traffic knowledge:

$$O_{\text{shared}}^{(l)} = \text{Sigmoid} \left(Z^{(l)} w_\Delta \right) \odot E_{\text{shared}} \left(Z^{(l)} \right), \quad (4)$$

where $w_\Delta \in \mathbb{R}^d$ is a projection vector, and $\odot$ represents element-wise multiplication with broadcasting over the feature dimension.

2) Specialized Experts for Heterogeneous Patterns. Simultaneously, a bank of N specialized experts $\{E_e\}_{e=1}^N$ is established to handle diverse traffic patterns. A routing network computes a probability distribution over these experts, selecting the Top- k most relevant experts to process each token. The routing scores $S^{(l)}$ and the sparse routing weights $\tilde{S}^{(l)}$ are computed as:

$$S^{(l)} = \text{Softmax} \left(Z^{(l)} W_* \right), \quad \tilde{S}_{ue}^{(l)} = \begin{cases} S_{ue}^{(l)}, & e \in \text{Top}_k(S_u^{(l)}) \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

where $W_* \in \mathbb{R}^{d \times N}$ projects the input into the expert embedding space, $S^{(l)} \in \mathbb{R}^{T \times N}$ contains per-token logits over experts, and $\tilde{S}^{(l)} \in \mathbb{R}^{T \times N}$ is a sparse routing matrix in which only the top- k entries per token (per row) are retained and the remaining entries are set to zero. This sparse routing mechanism ensures that, for each token, only a small number of experts are activated according to the current traffic semantics. The output of the sparse branch is the weighted sum of the activated experts:

$$O_{\text{special}}^{(l)} = \sum_{e=1}^N \tilde{S}_{ue}^{(l)} \odot E_e \left(Z^{(l)} \right), \quad (6)$$

where $\tilde{S}_{:,e}^{(l)}$ denotes the routing weights for expert E_e across all tokens and the multiplication $\odot$ is broadcast along the feature dimension. By setting $k \ll N$ (e.g., $k = 2$ in this work), *Traffic-MoE* selectively activates parameter subspaces relevant to the specific attack or protocol phase currently being processed.

For the implementation of the experts, both the shared expert and the specialized experts follow a SwiGLU-based gated feed-forward design. Given an input Z , an expert parameterized by $\{W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}\}$ computes:

$$E(Z) = (\text{SiLU}(ZW_{\text{gate}}) \odot (ZW_{\text{up}})) W_{\text{down}}, \quad (7)$$

where $\text{SiLU}(\cdot)$ is the Sigmoid Linear Unit. Note that the shared expert retains a full intermediate dimension $d' \gg d$ for capturing universal protocol patterns (i.e., $W_{\{\text{gate}, \text{up}\}} \in \mathbb{R}^{d \times d'}$ and $W_{\text{down}} \in \mathbb{R}^{d' \times d}$), whereas specialized experts use a reduced intermediate dimension (d'/k) to balance the number of experts and computation (i.e., $W_{\{\text{gate}, \text{up}\}} \in \mathbb{R}^{d \times \frac{d'}{k}}$ and $W_{\text{down}} \in \mathbb{R}^{\frac{d'}{k} \times d}$). The weights of different specialized experts are mutually independent.

The final output of the MoE layer integrates protocol-invariant features from the shared expert and the instance-specific features from the specialized experts, added to the residual stream:

$$H^{(l)} = \tilde{H}^{(l)} + \left(O_{\text{shared}}^{(l)} + O_{\text{special}}^{(l)} \right). \quad (8)$$

To prevent routing collapse, where the routing network converges to utilizing only a few experts, leaving others under-trained, we incorporate an auxiliary **load-balancing loss**, $\mathcal{L}_{\text{aux}}$. For a given layer l , we compute the loss based on the batch-wise statistics of expert activation. Let $\text{Load}_e^{(l)}$ be the fraction of tokens in the batch assigned to expert e , and $\text{Prob}_e^{(l)}$ be the average routing probability for expert e across the batch. The load-balancing loss for layer l is:

$$\mathcal{L}_{\text{aux}}^{(l)} = N \sum_{e=1}^N \text{Load}_e^{(l)} \cdot \text{Prob}_e^{(l)}, \quad \mathcal{L}_{\text{aux}} = \frac{1}{L} \sum_{l=1}^L \mathcal{L}_{\text{aux}}^{(l)}. \quad (9)$$

The total auxiliary loss $\mathcal{L}_{\text{aux}}$ is the average of these penalties across all L layers. This constraint ensures a uniform distribution of expert utilization at the batch level, maximizing the effective parameter capacity of the model while maintaining sparse inference efficiency.

4.3 Pre-training and Fine-tuning

To endow *Traffic-MoE* with both broad traffic understanding and task-specific discriminative ability, we adopt a two-stage training framework consisting of large-scale self-supervised pre-training followed by lightweight few-shot fine-tuning. The pre-training stage exposes the backbone to massive unlabeled traffic data, enabling it to learn protocol-aware and temporally structured representations through autoregressive next-token prediction and balanced sparse-expert activation. Fine-tuning then adapts these general representations to downstream traffic security applications by employing targeted data augmentation, flow-level semantic pooling, and hierarchical optimization. Together, these two stages allow *Traffic-MoE* to scale in capacity while remaining efficient and robust across diverse traffic security scenarios.

4.3.1 Autoregressive Pre-training: Next Token Prediction. Real-world networks generate massive amounts of unlabeled packet flows but

only limited annotated intrusion or application data, making pre-training essential for learning generalizable traffic representations. We therefore adopt a self-supervised autoregressive Next Token Prediction (NTP) objective to exploit this abundant unlabeled traffic. Because network communication is inherently sequential and governed by protocol-driven state transitions, predicting the subsequent token from historical context enables the model to learn temporal dependencies (e.g., TCP handshake timing) and structural regularities (e.g., TLS record header formats) even in encrypted flows. This process equips *Traffic-MoE* with a unified understanding of network behaviors that can be effectively adapted during downstream fine-tuning. Figure 2 illustrates the overall pre-training pipeline, where Traffic2Token representation, causal attention, and sparse experts interact to form a scalable traffic foundation model.

Given a flow token sequence $X = \{x_1, x_2, \dots, x_T\}$, *Traffic-MoE* learns to maximize the likelihood of the next token x_t conditioned on its historical context $X_{<t} = \{x_1, x_2, \dots, x_{t-1}\}$. The prefix $X_{<t}$ is encoded through L -layer *Traffic-MoE* backbone to produce hidden states $H_{<t}^{(L)} = \{h_1^{(L)}, \dots, h_{t-1}^{(L)}\}$. The final state $h_{t-1}^{(L)} \in \mathbb{R}^d$ summarizes all contextual information up to position $t-1$ and is projected into the vocabulary space to obtain the probability distribution of next-token prediction:

$$p_\theta(x_t | X_{<t}) = \text{Softmax}\left(h_{t-1}^{(L)} \cdot W_{\text{vocab}}\right), \quad (10)$$

where $W_{\text{vocab}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ maps hidden states to vocabulary logits. To optimize the model parameters θ , we employ the negative log-likelihood loss function $\mathcal{L}_{\text{NP}}$. For a mini-batch of B flows, this is calculated as the average loss over all valid tokens in the batch:

$$\mathcal{L}_{\text{NP}} = -\frac{1}{B \cdot T} \sum_{b=1}^B \sum_{t=1}^T \log p_\theta\left(x_t^{(b)} | X_{<t}^{(b)}\right). \quad (11)$$

Finally, as MoE activates only a subset of experts per token, balanced expert utilization becomes a key requirement in pre-training. We incorporate load-balancing term $\mathcal{L}_{\text{aux}}$ to prevent routing collapse and promote specialization across heterogeneous traffic behaviors. The overall pre-training objective is:

$$\mathcal{L}_{\text{pretrain}} = \mathcal{L}_{\text{NP}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}, \quad (12)$$

where λ_{aux} controls the strength of load-balancing regularization. Minimizing this composite loss enables *Traffic-MoE* to acquire rich, protocol-aware, and temporally grounded representations of traffic flows, while simultaneously encouraging efficient and diverse expert activation. As a result, the pre-trained backbone captures both universal traffic patterns and domain-specific subtleties, providing a strong foundation for downstream tasks such as intrusion detection, service classification, and VPN/Tor traffic analysis.

4.3.2 Few-shot Fine-tuning for Traffic Analysis. Downstream traffic analysis tasks such as malware traffic detection, service classification and C2 behavior identification typically face label scarcity and highly imbalanced class distributions. To effectively adapt *Traffic-MoE* to these data-constrained scenarios, we introduce a fine-tuning strategy tailored for few-shot learning. This strategy integrates temporal-aware data augmentation, flow-level representation aggregation, and hierarchical optimization to achieve robust performance under limited supervision.

Table 1: Overview of datasets used in pre-training and fine-tuning phases.

Dataset	Domain	Original Classes	Original Scale [*] (Record)	Key Protocols	Pre-training		Fine-tuning			
					Included	Size (Sessions)	Included	Downstream Task	Size (Sessions)	Classes
USTC-TFC2016	Malware Traffic	20	0.7M rows	FTP, SMB, HTTP, DNS, MySQL, ...	✓	203.2k	✗	-	-	-
UNSW-NB15	Network Intrusion	10	2.5M rows	HTTP, ICMP, FTP, SSH, DNS, ...	✓	828.3k	✗	-	-	-
CICIoT2023	IoT Security	34	46.7M rows	MQTT, HTTP, DNS, SSH, ARP, ...	✓	892.5k	✓	IoT Attack Detection	255.2k	34
CICIoMT2024	IoMT Security	19	8.8M rows	Wi-Fi, MQTT, HTTP, DNS, SSH, ...	✓	439.4k	✓	IoMT Attack Detection	107.3k	19
ISCXVPN2016(NonVPN)	Unencrypted Traffic	14	298.9k sessions	OpenVPN, BitTorrent, SFTP, FTPS, ...	✓	160.2k	✓	NonVPN Traffic Detection	23.9k	6
ISCXVPN2016(Mixed)	Encrypted VPN Traffic			HTTPS, SMTP/S, POP3/S, ...	✗	-	✓	Mixed Traffic Detection	26.2k	7
ISXCTor2016(NonTor)	Unencrypted Traffic	8	53.7k sessions	Tor, BitTorrent, HTTPS, SMTP, ...	✗	-	✓	NonTor Traffic Detection	51.5k	8
ISXCTor2016(Tor)	Encrypted Tor Traffic			POP3, SFTP, FTPS, ...	✗	-	✓	Tor Traffic Detection	2.0k	8

^{*} Note: The unit "rows" refers to pre-processed data records (e.g., packet windows or flow images) as defined in the original datasets, whereas "sessions" refers to complete bidirectional flows reconstructed from raw PCAP traces.

Network sessions vary widely in duration. DNS queries may finish within milliseconds, whereas video streaming or VPN tunnels can persist for minutes or even hours. This discrepancy leads to a long-tailed distribution of flow lengths and extremely limited samples for certain classes. To mitigate this imbalance, we apply Temporal Traffic Slicing, which divides long flows into multiple fixed-duration sub-flows that inherit the original label. The resulting augmented dataset increases sample diversity and enables the model to capture fine-grained temporal dynamics such as periodic beacons, idle intervals, and bursty transmissions that are often diluted when using full-session representations. This augmentation maximizes the utility of sparse session-level examples and is particularly effective in few-shot scenarios.

For supervised adaptation, we aggregate token-level representations from the final backbone layer into a unified flow-level embedding. Instead of relying on a single special token (e.g., [PD] or [END]), which may overlook mid-session structural semantics, we apply mean pooling aggregation over all valid tokens and compute class probabilities through a MLP-based prediction head:

$$p_{\theta}(y | X) = \text{Softmax}\left(\text{MLP}\left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t^{(L)}\right)\right). \quad (13)$$

For a mini-batch of B flow token sequences $\{X^{(b)}\}_{b=1}^B$ with labels $\{y_b\}_{b=1}^B$, the supervised loss is the batch-averaged cross-entropy:

$$\mathcal{L}_{\text{TC}} = -\frac{1}{B} \sum_{b=1}^B \log p_{\theta}(y_b | X^{(b)}). \quad (14)$$

To preserve the sparse-expert structure learned during fine-tuning, we retain the MoE load-balancing term $\mathcal{L}_{\text{aux}}$, yielding the overall fine-tuning objective:

$$\mathcal{L}_{\text{finetune}} = \mathcal{L}_{\text{TC}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}. \quad (15)$$

By minimizing this composite loss, *Traffic-MoE* can precisely adapt the general traffic knowledge learned during pre-training to downstream classification tasks while efficiently leveraging its powerful MoE architecture, thereby continuously improving its classification performance in specific traffic security applications.

Finally, a further consideration during fine-tuning is avoiding catastrophic forgetting of pre-trained knowledge. We therefore employ a Layer-wise Learning Rate Decay (LLRD) schedule that aligns with the hierarchical structure of the *Traffic-MoE* backbone. Shallower layers, which encode general byte-level patterns and packet-structure regularities, are updated conservatively with smaller

learning rates, whereas deeper layers and the classification head adapt more quickly to task-specific semantics. For a backbone with L layers and base learning rate η_0 , the learning rate for layer l is:

$$\eta_l = \xi^{L-l} \cdot \eta_0, \quad (16)$$

where $\xi < 1$ is the decay factor. This training strategy preserves the universal traffic knowledge encoded during pre-training while enabling the deeper layers to adjust to the semantic and distributional characteristics of the downstream task.

Through the combination of temporal slicing, flow-level semantic pooling, sparse-expert regularization, and hierarchical optimization, *Traffic-MoE* achieves high sample efficiency and robust generalization in data-constrained traffic security scenarios. Even with limited labeled examples, *Traffic-MoE* adapts effectively to new domains and evolving threat environments while retaining the broad traffic knowledge learned during pre-training.

5 Experiment Evaluation

5.1 Datasets and Downstream Tasks

5.1.1 Pre-training Corpus. To facilitate the learning of universal and robust traffic representations across heterogeneous network environments, we construct a comprehensive pre-training corpus aggregating over **2 million unlabeled flows** from five authoritative public datasets (USTC-TFC2016 [48], UNSW-NB15 [32], CICIoT2023 [33], CICIoMT2024 [8], ISCXVPN2016(NonVPN) [13]). These datasets encompass a wide spectrum of encryption complexities, protocol diversities, and behavioral signatures. Unlike prior works restricted to domain-specific subsets, our corpus incorporates extensive *distributional diversity* by intertwining standard IT traffic (e.g., HTTP/DNS) with specialized OT protocols (e.g., MQTT/Zigbee) and heavily obfuscated tunnels (e.g., Tor circuits). This cross-domain dataset synthesis forces the model to learn universal and intrinsic traffic semantics rather than dataset-specific artifacts.

5.1.2 Fine-tuning Datasets and Downstream Tasks. During fine-tuning phase, we select four challenging public datasets to design three downstream tasks, systematically evaluating model's transfer learning capability from attack detection to service classification: 1) For **Tor/NonTor Service Classification**, we leverage ISCX-Tor2016 to classify 8 application services within separated "Tor" and "NonTor" traffic subsets, challenging the model to capture subtle flow-level fingerprints in extreme anonymization scenarios; 2) For **NonVPN/Mixed Service Classification**, we employ ISCXVPN2016 to construct two sub-tasks, "NonVPN" and "Mixed

Table 2: Comparison with state-of-the-art methods on different traffic detection tasks.

Method	Task 1: ISCTXor2016(NonTor)				Task 1: ISCTXor2016(Tor)				Task 2: ISCXVPN2016(NonVPN)			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.9468	<u>0.8587</u>	0.6934	0.7436	<u>0.8690</u>	<u>0.7921</u>	<u>0.7054</u>	<u>0.7254</u>	0.5794	0.6756	0.6782	0.6588
FlowPrint	0.9213	0.7408	0.7245	0.7138	0.4127	0.1907	0.2595	0.1738	0.5459	0.6283	0.5620	0.5617
FS-Net	0.9406	0.7841	0.6518	0.6972	0.6679	0.2008	0.2527	0.2161	0.5093	0.4952	0.5021	0.4971
ET-BERT	0.9653	0.8322	0.7706	0.7960	0.7932	0.4093	0.4677	0.4303	0.5502	0.6670	0.6852	0.6755
NetGPT	<u>0.9655</u>	0.8479	<u>0.7911</u>	<u>0.8134</u>	0.8506	0.7342	0.6805	0.6583	<u>0.6618</u>	<u>0.7405</u>	<u>0.7511</u>	<u>0.7457</u>
TrafficFormer	0.9554	0.7663	0.7596	0.7418	0.7975	0.6083	0.5984	0.5428	0.6045	0.6831	0.6970	0.6842
Traffic-MoE	0.9827	0.9222	0.8707	0.8900	0.9089	0.8942	0.7879	0.8072	0.7613	0.7866	0.8158	0.8005

Method	Task 2: ISCXVPN2016(Mixed)				Task 3: CICIoMT2024				Task 3: CICIoT2023			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.6249	0.7414	0.7355	0.7248	0.6929	0.7473	0.6783	0.6397	0.5266	0.5535	0.4471	0.4619
FlowPrint	0.5659	0.6033	0.6431	0.5952	0.0132	0.0373	0.1364	0.0501	0.4578	0.3166	0.2048	0.1989
FS-Net	0.5357	0.4617	0.4681	0.4631	0.5788	0.5319	0.5050	0.4533	0.4903	0.5265	0.4046	0.4094
ET-BERT	0.6106	0.7503	0.7549	0.7521	0.9769	0.5620	0.5271	0.5255	0.7455	0.5416	0.5172	0.5217
NetGPT	<u>0.6973</u>	<u>0.8003</u>	<u>0.8072</u>	<u>0.8034</u>	<u>0.8958</u>	<u>0.8326</u>	<u>0.8335</u>	<u>0.8300</u>	<u>0.7684</u>	<u>0.7756</u>	<u>0.6709</u>	<u>0.6962</u>
TrafficFormer	0.6373	0.7689	0.7285	0.7422	0.8912	0.7572	0.7835	0.7530	0.7636	0.7172	0.6411	0.6594
Traffic-MoE	0.7679	0.8306	0.8377	0.8332	0.9769	0.8849	0.8860	0.8839	0.8588	0.8007	0.7701	0.7824

(VPN/NonVPN)” service classification, assessing the restoration of service semantics under tunnel encapsulation; 3) For **IoT/IoMT Attack Detection**, we utilize *CICIoT2023* to identify 33 distinct attack behaviors (e.g., Mirai, DDoS), and *CICIoMT2024* to detect 18 targeted attacks, testing performance in sensitive healthcare environments involving protocols like MQTT. Detailed statistics are provided in Table 1 and Appendix A.

5.2 Comparison Methods and Parameter Setup

5.2.1 Comparison Methods. To precisely benchmark *Traffic-MoE*, we evaluate it against six SOTA baselines categorized into two paradigms. As core baselines representing the “pre-training and fine-tuning” paradigm, we select *ET-BERT* [26], *NetGPT* [30], and *TrafficFormer* [57] for comparison. Additionally, to verify the effectiveness of the pre-training phase, we include two machine learning (ML) methods (*AppScanner* [46], *FlowPrint* [47]) and one deep learning (DL) method (*FS-Net* [27]) for comparison. To ensure fairness, all comparison models are reproduced based on their official open-source code. Specifically, the pre-training methods utilize the exact same pre-training and fine-tuning datasets constructed in this work, adopting identical early stopping strategies during fine-tuning to ensure optimal convergence, while the ML/DL methods are trained directly on the constructed fine-tuning datasets. More details of comparison methods are presented in Appendix C.

5.2.2 Implementation Details and Parameter Configuration. All experiments are implemented in PyTorch 2.2 and executed on a high-performance cluster equipped with Intel Xeon Gold 5218R CPUs (32-Core) and NVIDIA A100 GPUs (40GB VRAM). The environment runs on Ubuntu 22.04 LTS with CUDA 11.7.

Data Preprocessing and Serialization: Raw PCAP traces are first segmented into independent session flows via SplitCap. To

reduce statistical noise, we filter out micro-flows with fewer than 3 packets, except in categories with extreme sample scarcity where all valid flows are retained. We then use *Traffic2Token* to serialize flows. Specifically, we extract 11 bytes of header features and the first $J = 40$ bytes of the payload per packet, and truncate each session to the first $K = 10$ packets to balance computational efficiency and early-stage detection. The resulting vocabulary size is $|V| = 65,541$.

Training Configuration: We utilize AdamW optimizer with a batch size of $B = 32$. **For pre-training phase**, the model is trained for 8 epochs with a learning rate of 3.0×10^{-4} . The loss function combines the Next Token Prediction loss and the auxiliary load-balancing loss, with the regularization coefficient set to $\lambda_{\text{aux}} = 0.02$. **For fine-tuning phase**, datasets are split into training, validation, and testing sets with an 8:1:1 ratio. We employ a LLRD schedule to preserve pre-trained knowledge, setting the decay factor $\xi = 0.9$ and the base learning rate $\eta_0 = 5.0 \times 10^{-5}$. The maximum training duration is set to 40 epochs, safeguarded via early stopping with a patience of 5 epochs monitoring the validation Macro-F1 score.

5.2.3 Evaluation Metrics. Given the inherent long-tail distribution and class imbalance in network traffic, standard accuracy is insufficient for objective evaluation. We therefore adopt a multi-dimensional evaluation protocol. For evaluating general performance, we select Macro-Precision (M-PR), Macro-Recall (M-RC), and Macro-F1 Score (M-F1) as core metrics. By calculating performance independently for each class before averaging, these macro-metrics treat minority classes (e.g., emerging threats) and majority classes equally, preventing results from being biased by high-volume benign traffic. For evaluating security effectiveness, we specifically report the False Negative Rate (FNR) and False Positive Rate (FPR). These metrics are critical for assessing the operational risks of the model, quantifying the trade-off between missed

threats and false alarms in real-world deployments. Please refer to Appendix D for the formal definitions of these evaluation metrics.

5.3 Comparison with State-of-the-Art Methods

Table 2 reports the overall performance of all methods across diverse traffic analysis scenarios. Overall, *Traffic-MoE* exhibits consistent superiority, achieving the highest ranking across all evaluation metrics. A cross-paradigm comparison reveals a significant performance gap between pre-training methods and traditional approaches. Specifically, statistical ML methods and scratch-trained DL model lag behind pre-trained methods by substantial margins in most tasks. This disparity highlights a fundamental challenge in modeling heterogeneous network traffic. Traditional methods relying on shallow statistical features (e.g., packet length distributions) struggle to capture complex protocol behaviors obscured by encryption. Similarly, training deep networks from scratch fails to infer intricate protocol state transitions and temporal burst patterns given the scarcity of labeled attack samples. In contrast, by leveraging self-supervised pre-training on a massive unlabeled traffic corpus, *Traffic-MoE* internalizes the intrinsic “grammar” of network protocols (e.g., handshake sequences and payload structures), providing a semantic-rich initialization that significantly reduces dependency on task-specific labeled traffic data.

In the highly obfuscated Tor encrypted scenarios, *Traffic-MoE* demonstrates exceptional robustness. As shown in the ISCTXor2016 results, *Traffic-MoE* achieves Macro-F1 scores of 0.8900 (NonTor) and 0.8072 (Tor), outperforming the strongest baseline by 9.42% and 11.28%, respectively. Notably, existing strong baselines such as ET-BERT suffer from noticeable performance degradation in Tor traffic. This decline can be attributed to the Tor protocol’s multi-layer encryption and fixed-size cell padding mechanisms, which homogenize packet lengths and introduce significant structural noise, thereby confusing dense architectures that process all tokens uniformly. *Traffic-MoE* overcomes this challenge through its sparse MoE architecture. The dynamic routing mechanism allows specific experts to specialize in decoupling valid behavioral fingerprints (e.g., burst patterns) from padding noise, thereby maintaining high discriminative power even under extreme anonymity constraints.

Similarly, in VPN-encapsulated environments, *Traffic-MoE* effectively penetrates protocol obfuscation to identify underlying services. On the ISCXVPN2016 dataset, particularly in the “Mixed Traffic” detection task, *Traffic-MoE* outperforms the generative pre-trained model NetGPT by 10.12% in accuracy and 3.71% in Macro-F1. While traditional methods like AppScanner struggle due to the masking of superficial header features by VPN tunneling, *Traffic-MoE* succeeds by modeling long-range contextual dependencies. Through the self-attention mechanism, it captures intrinsic flow evolution patterns that remain invariant across tunnel encapsulation, thus resisting the interference of outer protocol headers.

Regarding attack detection, *Traffic-MoE* exhibits critical advantages in identifying malicious activities within massive background traffic. On the CICIoT2023 dataset, *Traffic-MoE* improves accuracy by 11.76% compared to the runner-up. More importantly for security defense, it achieves a Macro-Recall improvement of 3.79%, which directly translates to a significantly lower FNR. We observe that traditional methods fail to maintain performance in this scenario

Table 3: Detection performance of *Traffic-MoE* on different traffic categories (merged) in CICIoMT2024 and CICIoT2023.

	Merged Category	Num.#Test	Recall	FNR	FPR
CICIoMT2024	Benign	130	0.9462	0.0538	0.0011
	DDoS	4193	0.9955	0.0045	0.0046
	DoS	4075	0.9931	0.0069	0.0029
	Malformed	45	0.9333	0.0667	0.0001
	Recon	2229	0.9928	0.0072	0.0013
	Spoofing	67	0.7015	0.2985	0.0019
CICIoT2023	Benign	3534	0.8882	0.1118	0.0314
	BruteForce	265	0.7925	0.2075	0.0008
	DDoS	10775	0.9426	0.0574	0.0330
	DoS	4000	0.9785	0.0215	0.0021
	Mirai/Botnet	663	0.6591	0.3409	0.0084
	Recon	3754	0.8426	0.1574	0.0289
	Spoofing	2000	0.8250	0.1750	0.0125
	WebAttack	527	0.6831	0.3169	0.0045

due to the severe class imbalance and the presence of diverse attack variants. *Traffic-MoE* addresses this by internalizing anomalous patterns during the pre-training phase, enabling it to generalize from common attacks to unseen variants and effectively mitigating the long-tail distribution issue inherent in real-world threat detection.

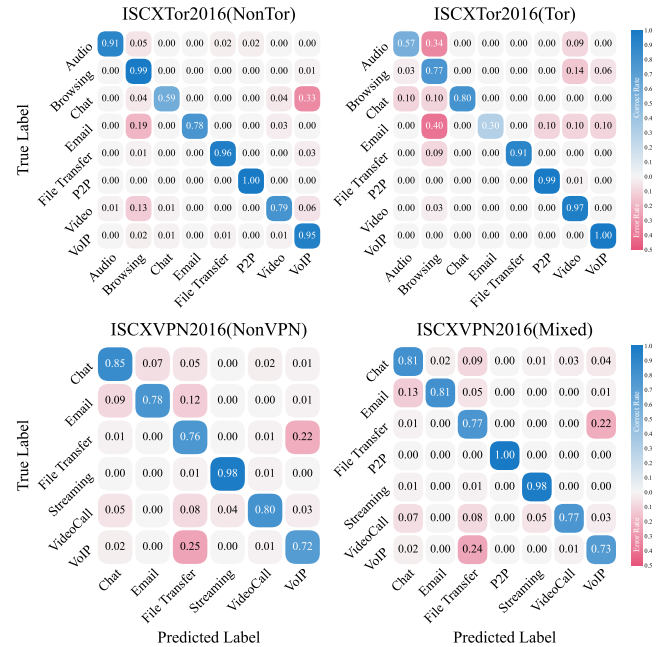


Figure 5: Performance confusion matrices of *Traffic-MoE* across different service classification tasks.

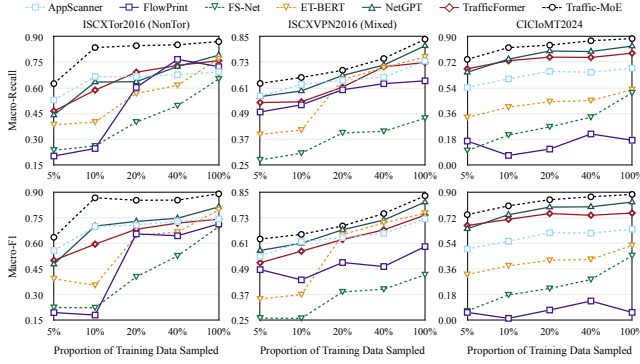


Figure 6: Performance evaluation in few-shot supervision.

5.4 Efficacy and Robustness Analysis

To investigate the performance boundaries of *Traffic-MoE* in real-world security scenarios, we conduct an in-depth analysis of fine-grained results, as shown in Figure 5 and Table 3.

5.4.1 Intrusion Detection: Minimizing Security Risks. In the context of network defense, the metrics for the *Benign* category serve as global indicators for system reliability: the **Benign FPR** represents the *Overall Attack Miss Rate* (attacks misclassified as benign), while the **Benign FNR** represents the *False Alarm Rate* (benign traffic misclassified as attacks). *Traffic-MoE* demonstrates exceptional capability in preventing defense penetration, achieving a negligible Attack Miss Rate (Benign FPR) of 0.11% on the sensitive CICIoMT2024 dataset and a robust 3.14% on the massive CICIoT2023 dataset. This implies that the vast majority of threats are successfully intercepted. Conversely, the False Alarm Rate (Benign FNR) is observed at 11.18% in IoT scenarios. This reflects a “Fail-Secure” design philosophy: given the intrinsic behavioral overlap between aggressive benign polling and malicious reconnaissance in IoT networks, the model prioritizes flagging ambiguous traffic for inspection rather than risking a breach. While volumetric attacks are detected with high precision, stealthier categories like *Spoofing* exhibit performance dips, suggesting that low-signature attacks remain a challenge for generalized representations.

5.4.2 Tor Anonymity: Countering Traffic Shaping. In adversarial anonymity scenarios, *Traffic-MoE* demonstrates a unique ability to counter traffic shaping mechanisms. These operations of fixed-size cell padding and randomized delays used by Tor protocol effectively distort the statistical feature space, leading to significant performance degradation for latency-sensitive services like *Audio* and transactional services like *Email* (as shown in Figure 5). However, *Traffic-MoE* exhibits resilience in interactive protocols such as *P2P* and *VoIP*, where detection accuracy remains high. This performance dichotomy indicates that the MoE architecture successfully shifts attention from eroded packet-length statistics to *sequence-level behavioral fingerprints* (e.g., handshake orders and interaction frequencies), effectively learning a “Protocol Grammar” that persists despite cryptographic obfuscation. Refer to Appendix E.2 for more analysis on *Audio/Email* service detection.

5.4.3 VPN Tunneling: Penetrating Encapsulation. Similarly, under VPN tunneling conditions (ISCXVPN2016), *Traffic-MoE* demonstrates strong resilience to protocol encapsulation. The confusion matrix reveals that misclassifications (e.g., between VoIP and File Transfer) are consistent across Non-VPN and Mixed scenarios, stemming from intrinsic feature similarity rather than tunnel noise. Crucially, for complex classes like Streaming and P2P, the model maintains near-perfect identification. This suggests that the self-attention mechanism effectively penetrates the outer tunnel headers to capture long-range flow dependencies, achieving semantic decoupling of the payload behavior from the transport encapsulation.

5.5 Comparison in Few-Shot Settings

In real-world cybersecurity operations, acquiring large-scale, high-quality labeled traffic is often cost-prohibitive due to the rapid evolution of attack variants. To evaluate robustness under *Label Scarcity*, we conduct a stratified few-shot evaluation on three representative datasets (*ISCTXTor2016-NonTor*, *ISCTXVPN2016-Mixed*, *CICIoMT2024*), scaling training data from 5% to 100%. The performance trajectories in Figure 6 reveal three distinct evolution paradigms. Scratch-trained DL methods (e.g., FS-Net) exhibit data-hungry collapse, where the lack of prior knowledge leads to parameter non-convergence and precipitous performance drops (over 85% decline) in extreme few-shot settings. In contrast, statistical ML methods (e.g., AppScanner) demonstrate stability but suffer from a heuristic representation ceiling, failing to scale performance with increased data due to the limited expressiveness of handcrafted features. Most notably, *Traffic-MoE* achieves **sample-efficient transfer**, significantly outperforming baselines at the 5% data extreme and often matching the full-data performance of competitors with only 10%-20% supervision, validating that the pre-trained MoE architecture successfully internalizes transfer-ready universal traffic semantics.

A deeper inspection of critical crossovers in Figure 6 highlights the architectural advantage of sparse experts over dense transformers. In complex encrypted scenarios (*ISCTXVPN2016*), we observe a watershed at the 20% data ratio: below this threshold, dense pre-trained baselines (e.g., ET-BERT) suffer significant degradation, dropping below even traditional ML methods. This indicates that dense architectures are prone to *overfitting and representation collapse* when fine-tuning data is insufficient to constrain their massive parameter space. Conversely, *Traffic-MoE* maintains SOTA performance, suggesting that *sparse expert activation mechanism effectively acts as a regularizer*. By activating only a subset of parameters ($k = 2$) per token, it reduces effective model complexity for specific tasks to prevent overfitting. Furthermore, in intrusion detection tasks (*CICIoMT2024*), *Traffic-MoE* maintains robust performance even when traditional methods collapse, demonstrating significant **generalization** potential for identifying emerging threats by effectively transferring learned protocol semantics to attack signatures.

5.6 Ablation Analysis

To verify the contribution of key components in *Traffic-MoE*, we conduct a multi-dimensional ablation study, with results summarized in Table 4. We analyze the architectural rationality from four perspectives. For a more detailed discussion on the theoretical implications of these ablation results, please refer to Appendix E.3.

Table 4: Ablation study results under different traffic detection tasks.

Ablation Model	Task 1: ISCTXor2016(NonTor)				Task 1: ISCTXor2016(Tor)				Task 2: ISCXVPN2016(NonVPN)			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
Traffic-MoE (Ours)	<u>0.9827</u>	<u>0.9222</u>	0.8707	<u>0.8900</u>	0.9089	0.8942	0.7879	0.8072	0.7613	<u>0.7866</u>	0.8158	0.8005
MoE→Dense	0.9831	0.9229	0.8655	0.8904	<u>0.9038</u>	0.8656	0.7638	0.7767	0.7178	0.7570	0.7900	0.7713
w/o PT	0.9643	0.7753	0.7271	0.7481	0.8304	0.7071	0.6392	0.6311	0.6062	0.6327	0.6095	0.6153
(MoE→Dense) w/o PT	0.9674	0.8064	0.8054	0.7938	0.8506	0.7056	0.6541	0.6564	0.6074	0.6334	0.6533	0.6389
w/o $\mathcal{L}_{aux}$	0.9800	0.8786	<u>0.8658</u>	0.8718	0.8962	<u>0.8734</u>	<u>0.7683</u>	<u>0.7913</u>	<u>0.7370</u>	0.7900	0.7990	<u>0.7929</u>
w/ Header Only	0.9750	0.8776	0.7983	0.8290	0.8759	0.7721	0.6385	0.6492	0.7308	0.7665	0.7846	0.7715
w/ Payload Only	0.9517	0.8935	0.7509	0.8055	0.8127	0.7597	0.7106	0.6996	0.4967	0.6436	0.4450	0.4472

Ablation Model	Task 2: ISCXVPN2016(Mixed)				Task 3: CICIoMT2024				Task 3: CICIoT2023			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
Traffic-MoE (Ours)	0.7679	0.8306	0.8377	0.8332	<u>0.9769</u>	<u>0.8849</u>	<u>0.8860</u>	<u>0.8839</u>	<u>0.8588</u>	0.8007	<u>0.7701</u>	<u>0.7824</u>
MoE→Dense	<u>0.7600</u>	<u>0.8101</u>	0.8302	<u>0.8193</u>	0.9790	0.8957	0.8947	0.8946	0.8594	0.8090	0.7765	0.7894
w/o PT	0.6255	0.6883	0.6865	0.6854	0.9685	0.8110	0.8236	0.8154	0.7833	0.6757	0.6584	0.6640
(MoE→Dense) w/o PT	0.6111	0.6958	0.6680	0.6770	0.9728	0.8350	0.8346	0.8294	0.7867	0.7054	0.6586	0.6741
w/o $\mathcal{L}_{aux}$	0.7353	0.8064	0.8205	0.8107	0.9767	0.8674	0.8725	0.8692	0.8575	0.8116	0.7630	0.7819
w/ Header Only	0.7425	0.8050	0.8150	0.8089	0.9263	0.8444	0.8539	0.8481	0.8395	0.7679	0.7211	0.7358
w/ Payload Only	0.5199	0.7147	0.5382	0.5648	0.4879	0.4603	0.4459	0.4210	0.3965	0.5782	0.3314	0.3710

5.6.1 Effectiveness of MoE Architecture and Sparse Activation. When comparing *Traffic-MoE* against a parameter-matched dense baselines (“MoE→Dense”) where MoE layers are replaced by standard FFNs, our method achieves competitive or superior performance while maintaining sparse inference efficiency (see Figure 7). Notably, in complex obfuscated scenarios such as *ISCTXor2016(Tor)* and *ISCXVPN2016(Mixed)*, *Traffic-MoE* surpasses the dense counterpart by **3.93%** and **1.69%** in Macro-F1, respectively. This confirms that the “expert specialization” mechanism effectively decouples heterogeneous traffic patterns via dynamic routing, which represents a capability that dense architectures struggle to attain.

5.6.2 Necessity of Pre-training. To evaluate the contribution of pre-training, we construct a “w/o PT” ablation model trained from scratch. The results explicitly validate the necessity of pre-training, as evidenced by a substantial performance decline across all tasks (e.g., a **23.14%** Macro-F1 drop on *ISCXVPN2016*). This confirms that universal traffic semantics acquired via large-scale unsupervised pre-training provide a critical initialization foundation. Crucially, the MoE architecture exhibits more severe degradation than the dense baseline in the absence of pre-training (e.g., **21.80%** vs. **15.49%** drop in the Tor scenario). This disparity underscores the router’s intrinsic dependency on a robust, pre-established semantic feature space to converge towards optimal expert allocation.

5.6.3 Role of Load Balancing Loss. To verify the impact of load balancing loss $\mathcal{L}_{aux}$, we evaluate an ablation model excluding this term, observing a general Macro-F1 decline of **0.06%–2.70%**. This result confirms that $\mathcal{L}_{aux}$ is critical for preventing “expert collapse”, where the router converges to local optima by directing traffic to a few over-utilized (dominant) experts. Such imbalance causes the model to degenerate into a sub-optimal dense architecture. Consequently, $\mathcal{L}_{aux}$ is essential for enforcing balanced expert activation and maximizing the utilization of the model’s overall capacity.

5.6.4 Complementarity of Input Representations. To evaluate the contributions of different input modalities, we compare ablation models using only header features (w/ Header Only) and only payload (w/ Payload Only). We observe a divergence in feature dominance. While header features dominate in attack detection (e.g., *CICIoT2023*), payload features are superior in Tor scenarios (outperforming headers by **5.04%** Macro-F1) due to the erosion of statistical header fingerprints by fixed-size cell padding. *Traffic-MoE* achieves optimal performance by dynamically fusing these modalities, compensating for single-view defects via multi-view decision fusion.

5.7 Inference Efficiency Analysis

To rigorously assess efficiency, we conduct an inference efficiency analysis of *Traffic-MoE* against a parameter-matched dense variant (“MoE→Dense”) and five other competitors on *CICIoMT2024* across varying concurrency loads (batch size $B \in \{8, \dots, 64\}$). As shown in Figure 7, *Traffic-MoE* demonstrates a dominant efficiency advantage that amplifies non-linearly with increasing traffic load. Specifically, under high-load conditions ($B = 64$), *Traffic-MoE* achieves a **41.57%** improvement in throughput and a **29.36%** reduction in latency compared to the dense variant. This significant performance gain stems from the decoupling of model capacity and inference FLOPs. By dynamically routing tokens to only top-2 experts, *Traffic-MoE* reduces computational complexity from $O(N_{total})$ to $O(N_{active})$, effectively bypassing approximately 40% of redundant computation. Furthermore, compared to the runner-up NetGPT, the throughput gain widens to **91.62%**, accompanied by a **38.72%** saving in peak GPU memory. This is primarily attributed to the *activation sparsity* of the MoE architecture, which reduces the memory usage of intermediate states during forward propagation, thereby alleviating resource bottlenecks that hinder foundation model deployment on throughput-sensitive security gateways.

5.8 Visualization Analysis of Expert Activation

To deconstruct the internal decision logic of the sparse MoE, we visualize average expert activation probability across layers under varying regularization constraints (w/ vs. w/o $\mathcal{L}_{aux}$) and training phases (pre-training vs. fine-tuning), as shown in Figure 8.

First, $\mathcal{L}_{aux}$ is pivotal in maintaining expert capacity. In the absence of $\mathcal{L}_{aux}$, expert activation states exhibit distinct collapse, characterized by the high-frequency activation of a sparse set of dominant experts within a single layer. This causes the model to degenerate into a capacity-limited dense network. Conversely, $\mathcal{L}_{aux}$ effectively enforces a balanced distribution of computational load, ensuring full utilization of the sparse parameter space.

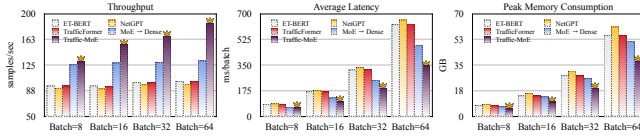


Figure 7: Inference efficiency comparison among different model architectures on CICIoMT2024.

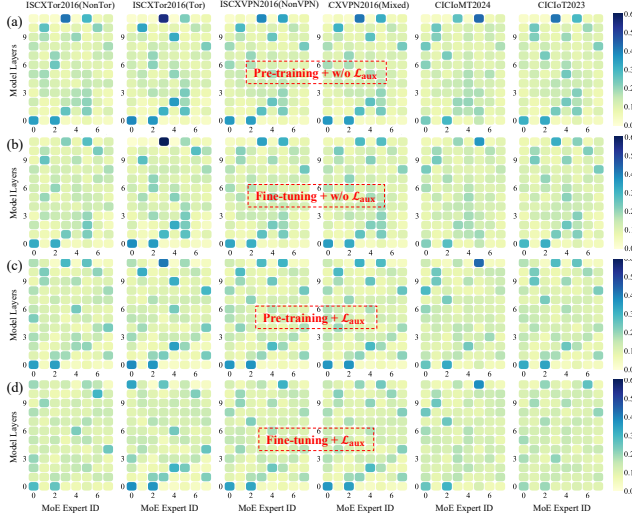


Figure 8: Expert Activation Across Model Layers.

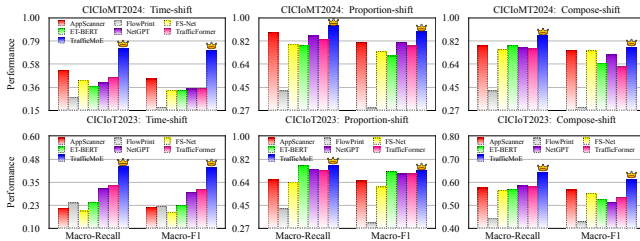


Figure 9: Performance under different distribution shifts.

Then the evolution of routing patterns from pre-training to fine-tuning provides empirical evidence for knowledge transfer and specialization. During pre-training, routers maintain a broad and relatively uniform activation distribution across all testing sets, indicating that the model primarily relies on a task-agnostic universal grammar acquired from large-scale corpora for inference. After fine-tuning, however, the activation patterns undergo a pronounced task-driven reconfiguration. Guided by specific task gradients, the routing strategy exhibits a significant clustering shift, directing traffic towards specific subsets of experts. This behavior suggests that pre-trained experts have successfully transformed into domain experts specialized in processing specific downstream features (e.g., obfuscation artifacts or attack signatures). This evolutionary trajectory demonstrates that *Traffic-MoE* can effectively leverage general priors as an initialization foundation and achieve specialized allocation of computational resources through dynamic adjustment of routing logic, thereby maximizing discriminative accuracy on downstream tasks while preserving generalization capability.

Furthermore, expert activation patterns exhibit distinct distribution characteristics across varying network depths. Shallow experts exhibit highly consistent activation patterns across all downstream tasks, indicating that they function as universal syntax parsers focused on extracting low-level, task-agnostic protocol features (e.g., TCP/IP header fields, TLS handshake sequences). Conversely, deep experts show similarity only between semantically related tasks (e.g., VPN/Tor, which involve encrypted tunnels), suggesting that they have converged upon abstract, high-level behavioral semantics (e.g., encryption entropy distributions, traffic burst patterns). Intermediate experts, however, exhibit significant distributional divergence across different tasks, reflecting the model undergoing rigorous semantic transition and feature disentanglement.

5.9 Evaluation under Distribution Shifts

Real-world network environments are highly dynamic, characterized by continuous temporal evolution and non-stationary class distributions. To evaluate whether *Traffic-MoE* can adapt to such “open-world” challenges without frequent retraining, we construct three OOD benchmarks: **Time-shift** (simulating concept drift over time), **Proportion-shift** (simulating intra-class distribution shift), and **Compose-shift** (simulating unseen attack sub-variants). Please refer to Appendix E.6 for more details. Figure 9 visualizes the performance comparison across these scenarios. **1) Resilience to Concept Drift:** The most significant performance gap appears in the Time-shift scenario. While existing competitors (e.g., FS-Net, ET-BERT) suffer catastrophic degradation due to temporal feature shifts, *Traffic-MoE* maintains a robust performance, outperforming the runner-up by 38.55%–60.18% across datasets. This suggests that the sparse expert architecture effectively disentangles time-invariant protocol semantics from transient statistical noise, preventing the model from overfitting to temporal shortcuts. **2) Stability under Structural Shifts:** In scenarios involving structural mutations (Proportion/Compose-shift), *Traffic-MoE* consistently achieves SOTA performance. Notably, in the Compose-shift task (where 50% of sub-classes are unseen during training), *Traffic-MoE* surpasses strong pre-trained competitors (e.g., NetGPT) by significant margins. High recall rates in these tasks indicate that the model

learns generalized behavioral abstractions rather than memorizing specific signatures of seen sub-classes. In summary, *Traffic-MoE* exhibits superior generalization and robustness against distribution shifts, validating its viability for sustained operation in evolving network defense systems.

6 Conclusion

This work bridges the gap between advanced representation learning and practical network protection by introducing *Traffic-MoE*, an efficient sparse foundation model. By synergizing self-supervised pre-training with a dynamic Mixture-of-Experts architecture, *Traffic-MoE* effectively decouples model capacity from inference overhead. Extensive evaluations demonstrate that *Traffic-MoE* establishes new state-of-the-art performance across heterogeneous security tasks including intrusion detection and encrypted service classification while significantly reducing inference latency compared to dense competitors. Furthermore, its superior robustness and generalization under few-shot supervision and distribution shifts confirms that sparse foundation models offer a scalable, resilient, and efficient paradigm for next-generation real-time network defense.

References

- [1] Mohammed Ali Al-Garadi, Amr Mohamed, Abdulla Khalid Al-Ali, Xiaojiang Du, Ihsan Ali, and Mohsen Guizani. 2020. A survey of machine and deep learning methods for internet of things (IoT) security. *IEEE communications surveys & tutorials* 22, 3 (2020), 1646–1685.
- [2] Blake Anderson and David McGrew. 2016. Identifying encrypted malware traffic with contextual flow data. In *Proceedings of the 2016 ACM workshop on artificial intelligence and security*. 35–46.
- [3] Blake Anderson, Subharthi Paul, and David McGrew. 2018. Deciphering malware's use of TLS (without decryption). *Journal of Computer Virology and Hacking Techniques* 14, 3 (2018), 195–211.
- [4] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J Alex Halderman, Luca Invernizzi, Michalis Kallitsis, et al. 2017. Understanding the mirai botnet. In *26th USENIX security symposium (USENIX Security 17)*. 1093–1110.
- [5] Ahmad Azab, Mahmoud Khasawneh, Saed Alrabaa, Kim-Kwang Raymond Choo, and Maysa Sarsour. 2024. Network traffic classification: Techniques, datasets, and challenges. *Digital Communications and Networks* 10, 3 (2024), 676–692.
- [6] Alireza Bahramali, Ardavan Bozorgi, and Amir Houmansadr. 2023. Realistic website fingerprinting by augmenting network traces. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1035–1049.
- [7] Juan Camilo Correa Chica, Jenny Cuatindioy Imbachi, and Juan Felipe Botero Vega. 2020. Security in SDN: A comprehensive survey. *Journal of Network and Computer Applications* 159 (2020), 102595.
- [8] Sajjad Dadkhah, Euclides Carlos Pinto Neto, Raphael Ferreira, Reginald Chukwu, Molokwu, Somayeh Sadeghi, and Ali A Ghorbani. 2024. CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT. *Internet of Things* 28 (2024), 101351.
- [9] Roger Dingledine, Nick Mathewson, and Paul Syverson. 2004. Tor: The second-generation onion router. (2004).
- [10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [11] Igor Fosić, Drago Žagar, Krešimir Grgić, and Višnja Križanović. 2023. Anomaly detection in NetFlow network traffic using supervised machine learning algorithms. *Journal of industrial information integration* 33 (2023), 100466.
- [12] Chuanpu Fu, Qi Li, Meng Shen, and Ke Xu. 2024. Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3659–3673.
- [13] Gerard Drapper Gil, Arash Habibi Lashkari, Mohammad Mamun, and Ali A Ghorbani. 2016. Characterization of encrypted and VPN traffic using time-related features. In *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP 2016)*. SciTePress Setúbal, Portugal, 407–414.
- [14] Jie Gui, Tuo Chen, Jing Zhang, Qiong Cao, Zhenan Sun, Hao Luo, and Dacheng Tao. 2024. A survey on self-supervised learning: Algorithms, applications, and future trends. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 12 (2024), 9052–9071.
- [15] Chaoqun Guo, Nan Wang, Yuanlin Sun, and Dalin Zhang. 2023. DTC: Addressing the long-tailed problem in intrusion detection through the divide-then-conquer paradigm. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 1319–1326.
- [16] Hong Ye He, Zhi Guo Yang, and Xiang Ning Chen. 2020. PERT: Payload encoding representation from transformer for encrypted traffic classification. In *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, 1–8.
- [17] Jordan Holland, Paul Schmitt, Nick Feamster, and Prateek Mittal. 2021. New directions in automated traffic analysis. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 3366–3383.
- [18] Amir Houmansadr, Chad Brubaker, and Vitaly Shmatikov. 2013. The parrot is dead: Observing unobservable network communications. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 65–79.
- [19] Hongxin Hu, Wonkyu Han, Gail-Joon Ahn, and Ziming Zhao. 2014. FLOW-GUARD: Building robust firewalls for software-defined networks. In *Proceedings of the third workshop on Hot topics in software defined networking*. 97–102.
- [20] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixture of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [21] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasnic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, et al. 2017. The quic transport protocol: Design and internet-scale deployment. In *Proceedings of the conference of the ACM special interest group on data communication*. 183–196.
- [22] Arash Habibi Lashkari, Gerard Draper Gil, Mohammad Saiful Islam Mamun, and Ali A Ghorbani. 2017. Characterization of tor traffic using time based features. In *International conference on information systems security and privacy*, Vol. 2. SciTePress, 253–262.
- [23] Hyunwoo Lee, Doowon Kim, and Yonghwi Kwon. 2021. TLS 1.3 in practice: How TLS 1.3 contributes to the internet. In *Proceedings of the Web Conference 2021*. 70–79.
- [24] Dmitry Lepikhin, Hyoukjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
- [25] Peiyang Li, Ye Wang, Qi Li, Zhuotao Liu, Ke Xu, Ju Ren, Zhiying Liu, and Ruilin Lin. 2023. Learning from limited heterogeneous training data: Meta-learning for unsupervised zero-day web attack detection across web domains. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1020–1034.
- [26] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. 2022. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference*. 633–642.
- [27] Chang Liu, Longtao He, Gang Xiong, Zigang Cao, and Zhen Li. 2019. FS-Net: A flow sequence network for encrypted traffic classification. In *IEEE INFOCOM 2019-IEEE Conference On Computer Communications*. IEEE, 1171–1179.
- [28] Ya Liu, Xiao Wang, Bo Qu, and Fengyu Zhao. 2024. ATVTSC: A Novel Encrypted Traffic Classification Method Based on Deep Learning. *IEEE Transactions on Information Forensics and Security* 19 (2024), 9374–9389.
- [29] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammadsadegh Saberian. 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing* 24, 3 (2020), 1999–2012.
- [30] Xuying Meng, Chungang Lin, Yequan Wang, and Yujun Zhang. 2023. Net-GPT: Generative pretrained transformer for network traffic. *arXiv preprint arXiv:2304.09513* (2023).
- [31] Andrew W Moore and Konstantina Papagiannaki. 2005. Toward the accurate identification of network applications. In *International workshop on passive and active network measurement*. Springer, 41–54.
- [32] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference (MilCIS)*. IEEE, 1–6.
- [33] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A Ghorbani. 2023. CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 23, 13 (2023), 5941.
- [34] Eva Papadogiannaki and Sotiris Ioannidis. 2021. A survey on encrypted network traffic analysis applications, techniques, and countermeasures. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–35.
- [35] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P Wellman. 2018. Sok: Security and privacy in machine learning. In *2018 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 399–414.
- [36] Eric Rescorla. 2018. *The transport layer security (TLS) protocol version 1.3*. Technical Report.
- [37] Martin Roesch et al. 1999. Snort: Lightweight intrusion detection for networks. In *Lisa*, Vol. 99. 229–238.

- [38] Matthew Roughan, Subhabrata Sen, Oliver Spatscheck, and Nick Duffield. 2004. Class-of-service mapping for QoS: a statistical signature-based approach to IP traffic classification. In *Proceedings of the 4th ACM SIGCOMM conference on Internet measurement*. 135–148.
- [39] Abid Saber, Belkacem Fergani, and Moncef Abbas. 2018. Encrypted traffic classification: Combining over-and under-sampling through a pca-svm. In *International conference on pattern analysis and intelligent systems (PAIS)*. IEEE, 1–5.
- [40] Yam Sharon, David Berend, Yang Liu, Asaf Shabtai, and Yuval Elovici. 2022. Tantra: Timing-based adversarial network traffic reshaping attack. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3225–3237.
- [41] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*. 1–19.
- [42] Meng Shen, Jinhe Wu, Ke Ye, Ke Xu, Gang Xiong, and Liehuang Zhu. 2025. Robust Detection of Malicious Encrypted Traffic via Contrastive Learning. *IEEE Transactions on Information Forensics and Security* (2025).
- [43] Meng Shen, Ke Ye, Xingtong Liu, Liehuang Zhu, Jiawen Kang, Shui Yu, Qi Li, and Ke Xu. 2022. Machine learning-powered encrypted network traffic analysis: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 25, 1 (2022), 791–824.
- [44] Robin Sommer and Vern Paxson. 2010. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE symposium on security and privacy*. IEEE, 305–316.
- [45] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [46] Vincent F Taylor, Riccardo Spolaor, Mauro Conti, and Ivan Martinovic. 2017. Robust smartphone app identification via encrypted network traffic analysis. *IEEE Transactions on Information Forensics and Security* 13, 1 (2017), 63–78.
- [47] Thijs Van Ede, Riccardo Bortolameotti, Andrea Continella, Jingjing Ren, Daniel J Dubois, Martina Lindorfer, David Choffnes, Maarten Van Steen, and Andreas Peter. 2020. Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In *Network and distributed system security symposium (NDSS)*, Vol. 27. 1–18.
- [48] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng. 2017. Malware traffic classification using convolutional neural network for representation learning. In *2017 International conference on information networking (ICOIN)*. IEEE, 712–717.
- [49] Guanglu Wei and Zhonghua Wang. 2021. Adoption and realization of deep learning in network traffic anomaly detection device design. *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 25, 2 (2021), 1147–1158.
- [50] Yonghui Wu. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016).
- [51] Diwen Xue, Reethika Ramesh, Arham Jain, Michaelis Kallitsis, J Alex Halderman, Jedidiah R Crandall, and Roya Ensafi. 2025. OpenVPN is open to VPN fingerprinting. *Commun. ACM* 68, 1 (2025), 79–87.
- [52] Yi Zeng, Huaxi Gu, Wenting Wei, and Yantao Guo. 2019. *Deep – Full – Range*: a deep learning based network encrypted traffic classification and intrusion detection framework. *IEEE Access* 7 (2019), 45182–45190.
- [53] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).
- [54] Haozhen Zhang, Le Yu, Xi Xiao, Qing Li, Francesco Mercaldo, Xiapu Luo, and Qixu Liu. 2023. TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. In *Proceedings of the ACM web conference*. 2066–2075.
- [55] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qianqian Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *the 27th Network and Distributed System Security Symposium (NDSS 2020)*.
- [56] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Yanhao Wang, Yijun Wang, Guan Gui, and Zhi Xue. 2023. Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5420–5427.
- [57] Guangmeng Zhou, Xiongwen Guo, Zhuotao Liu, Tong Li, Qi Li, and Ke Xu. 2025. Trafficformer: an efficient pre-trained model for traffic data. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1844–1860.
- [58] Jiajun Zhou, Wentao Fu, Hao Song, Shanqing Yu, and Qi Xuan. 2025. Multiview Correlation-Aware Network Traffic Detection on Flow Hypergraph. *IEEE Internet of Things Journal* 12, 24 (2025), 55922–55935.
- [59] Shitong Zhu, Shasha Li, Zhongjie Wang, Xun Chen, Zhiyun Qian, Srikanth V Krishnamurthy, Kevin S Chan, and Ananthram Swami. 2020. You do (not) belong here: detecting DPI evasion attacks with context learning. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*. 183–197.

A Fine-tuning Datasets and Downstream Tasks

In the fine-tuning phase, we select four challenging public datasets and design six downstream classification tasks. These tasks are intended to systematically evaluate the model's transfer learning capability across multiple dimensions, ranging from attack detection sensitivity to encrypted service identification rates.

1) IoT Attack Detection. Based on the *CICIoT2023* [33] dataset, this scenario simulates a complex network environment consisting of 105 real IoT devices. The task covers 33 distinct attack behaviors categorized into 7 major classes, including DDoS, Brute Force, Web attacks, and Mirai botnets. This task evaluates the model's capability for fine-grained classification of diverse attack flows amidst massive background traffic.

2) IoMT Attack Detection. Based on the *CICIoMT2024* [8] dataset, the data originates from a testbed containing 40 real/simulated medical devices, covering 18 targeted attacks across various protocols such as Wi-Fi and MQTT. Given the high sensitivity of healthcare environments, this task focuses on examining the model's detection performance in specialized medical domains against attacks exploiting proprietary protocol vulnerabilities.

3) VPN/NonVPN Service Classification. Based on the *IS-CXVPN2016* [13] dataset. This task is highly challenging as VPN technologies introduce multi-layer encryption and tunnel encapsulation, significantly altering traffic statistical characteristics. We construct two sub-tasks: "NonVPN Service Classification" and "Mixed Traffic (VPN and NonVPN) Service Classification". These aim to verify whether the model can effectively identify the underlying service behavioral patterns despite protocol obfuscation and strong encryption interference.

4) Tor/NonTor Anonymous Service Classification. Based on the *ISCTXTor2016* [22] dataset. The Onion Router (Tor) achieves high anonymity through multi-layer encryption and node hopping. We divide the dataset into two subsets, "Tor Traffic" and "NonTor Traffic", and classify 8 distinct application services within each. This task tests the model's efficacy in capturing subtle flow-level features and application-layer fingerprints in extreme anonymization scenarios.

B Pseudocode of Model Training

C Details of Comparison Methods

We compare *Traffic-MoE* against six representative baselines, categorized into traditional learning and pre-training paradigms:

C.1 Traditional ML/DL Methods

- **AppScanner** [46]: A classic statistical fingerprinting method. It extracts statistical features from encrypted traffic (e.g., packet lengths and inter-arrival time) and employs a Random Forest classifier to identify applications without payload inspection.
- **FlowPrint** [47]: A semi-supervised approach designed for open-world scenarios. It mines temporal correlations within traffic flows to construct a dynamic fingerprint library, enabling the identification of previously unseen applications.
- **FS-Net** [27]: An end-to-end deep learning architecture tailored for sequence modeling. It utilizes a Bidirectional Gated Recurrent Unit (Bi-GRU) in an encoder-decoder structure to learn latent

Algorithm 1 Pretraining and Fine-tuning of *Traffic-MoE*.

```

1: Input: batch of flow token sequences  $\{X^{(b)}\}_{b=1}^B$ , training mode
    $\star \in \{\text{PRETRAIN}, \text{FINETUNE}\}$ , labels  $\{y_b\}_{b=1}^B$  if FINETUNE;
2: hyperparameters: transformer depth  $L$ , number of experts  $N$ ,
   top- $k$  routing  $k$ , auxiliary weight  $\lambda_{\text{aux}}$ ;
3: Output: updated model (and prediction head if FINETUNE);
   /* Shared backbone: causal transformer with sparse MoE */
4: for  $b = 1$  to  $B$  do
5:   Initialize hidden state of token sequences:  $H^{(0,b)} \leftarrow X^{(b)}$ ;
6:   for  $l = 1$  to  $L$  do
7:     Compute causal self-attention  $O_{\text{attn}}^{(l,b)}$  via Eq. (1)~(2);
8:     Residual update via Eq. (3):  $\tilde{H}^{(l,b)} \leftarrow H^{(l-1,b)} + O_{\text{attn}}^{(l,b)}$ ;
9:     Apply sparse MoE to compute  $H^{(l,b)}$  via Eq. (4)~(8);
10:   end for
11: end for
12: Compute MoE load-balancing loss  $\mathcal{L}_{\text{aux}}$  via Eq. (9);
   /* Pretraining */
13: if  $\star$  is PRETRAIN then
14:   Compute autoregression loss  $\mathcal{L}_{\text{NP}}$  over batch via Eq. (11);
15:   Combine objectives:  $\mathcal{L}_{\text{pretrain}} \leftarrow \mathcal{L}_{\text{NP}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}$ ;
16:   Update backbone parameters by minimizing  $\mathcal{L}_{\text{pretrain}}$ ;
   /* Fine-tuning */
17: else
18:   For each flow  $X^{(b)}$ , perform mean pooling over final-layer
     tokens and classify to obtain  $p_{\theta}(y_b | X^{(b)})$  via Eq. (13);
19:   Compute classification loss  $\mathcal{L}_{\text{TC}}$  via Eq. (14);
20:   Combine objectives:  $\mathcal{L}_{\text{finetune}} \leftarrow \mathcal{L}_{\text{TC}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}$ ;
21:   Update backbone and prediction head by minimizing
      $\mathcal{L}_{\text{finetune}}$  with layer-wise learning-rate decay;
22: end if

```

representations directly from raw packet length sequences, bypassing manual feature engineering.

C.2 Pre-trained Foundation Models

- **ET-BERT** [26]: A pioneering method adapting the BERT architecture to encrypted traffic domain. It introduces two pre-training objectives—Masked Burst Model (MBM) and Same-origin Burst Prediction (SBP)—to learn contextual dependencies from massive unlabeled flow data.
- **NetGPT** [30]: The first framework applying Generative Pre-trained Transformers (GPT) to the traffic domain. It treats traffic flows as hexadecimal byte sequences and unifies traffic understanding and generation tasks, adapting to downstream tasks via prompt tuning.
- **TrafficFormer** [57]: A recent advancement building upon ET-BERT. It introduces a Same-Origin-Direction-Flow (SODF) multi-task objective to capture fine-grained packet directionality and sequence relationships more effectively.

Note on Implementation: To ensure a fair comparison of *model capacity* rather than data engineering tricks, we standardize the experimental setting. We exclude method-specific data augmentation strategies (e.g., randomization of header fields) used in the original papers and fine-tune all models on identical raw datasets.

D Formal Definitions of Evaluation Metrics

To objectively evaluate the performance of *Traffic-MoE*, we utilize a comprehensive set of metrics derived from the confusion matrix. For a specific class c within a total of C classes, let TP_c , FP_c , FN_c , and TN_c denote the counts of True Positives, False Positives, False Negatives, and True Negatives, respectively.

D.1 General Classification Metrics

The standard metrics—Precision (Pr), Recall (Rc), and F1-Score ($F1$)—are defined as follows:

$$\begin{aligned} \text{Precision}_c &= \frac{TP_c}{TP_c + FP_c}, & \text{Recall}_c &= \frac{TP_c}{TP_c + FN_c}, \\ F1_c &= 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \end{aligned} \quad (17)$$

To account for class imbalance, we employ the **Macro-averaged** metrics as the primary criteria, which calculates the arithmetic mean of the per-class metrics:

$$\text{Macro-}\#\text{Metric} = \frac{1}{C} \sum_{c=1}^C \#\text{Metric}_c. \quad (18)$$

D.2 Security-Oriented Metrics

For intrusion detection systems, the asymmetric impact of omitted threats and spurious alarms is a critical concern. Consequently, we additionally report the False Negative Rate (FNR) and False Positive Rate (FPR) to enable comprehensive evaluation. For a specific category c (e.g., an attack type), FNR_c represents the proportion of threats that evade detection; while FPR_c quantifies the false alarm rate, indicating the likelihood of non- c instances (including benign traffic) being erroneously flagged as c :

$$\text{FNR}_c = \frac{FN_c}{TP_c + FN_c} = 1 - \text{Recall}_c, \quad \text{FPR}_c = \frac{FP_c}{FP_c + TN_c}. \quad (19)$$

E More Analysis

E.1 Fine-grained Categories $\Rightarrow$ Merged Classes

Table 5 presents the mapping between fine-grained categories and merged classes in the *CICIoMT2024* and *CICIoT2023* datasets. Complementary to Figure 5, Figure 10 further exhibits the performance confusion matrices on two intrusion detection datasets: *CICIoMT2024* and *CICIoT2023*.

E.2 Performance Analysis on Audio and Email Service Identification in Tor Scenarios (As a Supplement to Sec. 5.4.2)

In the experimental evaluation on the *ISCXTor2016* dataset, we observe a performance dichotomy: while interactive protocols (e.g., P2P, VoIP) maintain high detection rates, latency-sensitive streams (*Audio*) and transactional services (*Email*) experience significant performance degradation. This phenomenon is not an architectural deficiency of *Traffic-MoE*, but rather a theoretical inevitability resulting from *Feature Collapse* induced by Tor’s traffic shaping mechanisms. We attribute this to three primary factors:

1) Erosion of Statistical Fingerprints. Traditional identification of streaming media (*Audio*) and short-burst messaging (*Email*)

Table 5: Mapping between fine-grained categories and merged classes in the *CICIoMT2024* and *CICIoT2023* datasets.

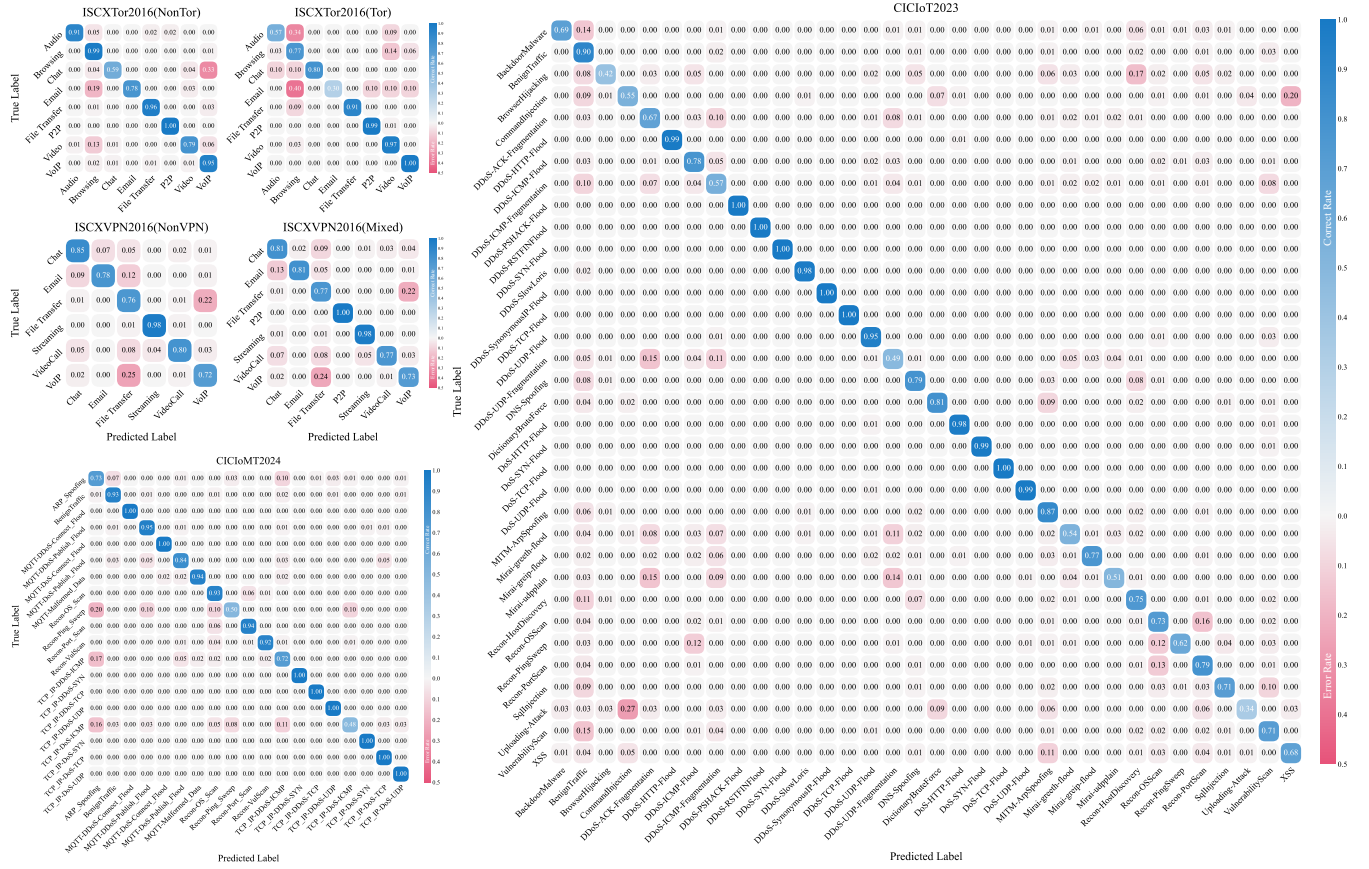
	Merged Categories	Fine-grained Categories
CICIoMT2024	Benign	BenignTraffic
	DDoS	MQTT-DDoS-Connect_Flood, MQTT-DDoS-Publish_Flood, TCP_IP-DDoS-ICMP, TCP_IP-DDoS-SYN, TCP_IP-DDoS-TCP, TCP_IP-DDoS-UDP
	DoS	MQTT-DoS-Connect_Flood, MQTT-DoS-Publish_Flood, TCP_IP-DoS-ICMP, TCP_IP-DoS-SYN, TCP_IP-DoS-TCP, TCP_IP-DoS-UDP
	Malformed	MQTT-Malformed_Data
	Recon	Recon-OS_Scan, Recon-Ping_Sweep, Recon-Port_Scan, Recon-VulScan
	Spoofing	ARP_Spoofing
CICIoT2023	Benign	BenignTraffic
	BruteForce	DictionaryBruteForce
	DDoS	DDoS-ACK-Fragmentation, DDoS-HTTP-Flood, DDoS-ICMP-Flood, DDoS-ICMP-Fragmentation, DDoS-PSHACK-Flood, DDoS-RSTFINFlood, DDoS-SYN-Flood, DDoS-SlowLoris, DDoS-SynonymousIP-Flood, DDoS-TCP-Flood, DDoS-UDP-Flood, DDoS-UDP-Fragmentation
	DoS	DoS-HTTP-Flood, DoS-SYN-Flood, DoS-TCP-Flood, DoS-UDP-Flood
	Mirai/Botnet	Mirai-greeth-flood, Mirai-greip-flood, Mirai-udpplain
	Recon	Recon-HostDiscovery, Recon-OSScan, Recon-PingSweep, Recon-PortScan, VulnerabilityScan
	Spoofing	DNS-Spoofing, MITM-Arpspoofing
	WebAttack	Backdoor/Malware, BrowserHijacking, CommandInjection, SqlInjection, Uploading-Attack, XSS

relies heavily on statistical side-channels: packet length distributions (payload size) and inter-arrival times (IAT). Tor enforces fixed-size cell padding (512 bytes) and multi-hop randomized delays (jitter). These mechanisms effectively homogenize the unique statistical distributions of *Audio* and *Email*. Specifically, the continuous, stable flow of *Audio* is transformed into a bursty sequence indistinguishable from bulk web downloading, while the handshake size characteristics of *Email* are masked by padding.

2) Low Information Density in Input Window. Our model utilizes the first $K = 10$ packets of a flow. For complex interactive protocols (VoIP/P2P), this window captures rich “Protocol Grammar”, such as authentication handshakes and signaling exchanges. However, *Audio* and *Email* protocols often exhibit low interaction density during the setup phase. In the first 10 packets, an *Audio* stream (often just establishing a TCP/TLS connection) lacks the distinct state transitions found in P2P. Once the statistical features are stripped by Tor, the remaining information within the input window is insufficient to form a unique semantic signature.

3) A “Many-to-One” Mapping Problem. From an information-theoretic perspective, Tor’s obfuscation creates a “Many-to-One” mapping where distinct application-layer behaviors (continuous streaming vs. bulk downloading vs. message sending) are mapped to an identical physical-layer representation (a sequence of fixed-size encrypted blocks with randomized timing). For the MoE’s Gating Network, the input feature vectors for *Audio*, *Email*, and *Browsing* become highly indistinguishable. Consequently, the router cannot learn a discriminative policy to dispatch these tokens to specialized experts, leading to the observed confusion in the results.

Summary: The failure of *Audio/Email* detection under Tor paradoxically validates the success of *Traffic-MoE*’s design. It confirms that our model has shifted its focus from superficial statistical features (which are easily destroyed) to deep sequential behavioral

Figure 10: Performance confusion matrices of *Traffic-MoE* across different traffic downstream tasks.

patterns (which persist in P2P/VoIP), proving robustness against traffic shaping where semantic structure remains intact.

E.3 Detailed Ablation Analysis

To quantify the contribution of key model components and verify the rationality of the architectural design, we conduct a multi-dimensional ablation analysis based on the experimental results summarized in Table 4. This section sequentially explores the effectiveness of the MoE architecture, the necessity of the pre-training strategy, the role of the auxiliary loss function, and the complementarity of input representations.

E.3.1 Effectiveness of MoE Architecture and Sparse Activation. To validate the benefits of the MoE architecture, we construct a dense ablation model with a comparable parameter count (denoted as “MoE→Dense”), where the MoE layers are replaced by standard FFN layers, and train it under identical conditions. Experimental results indicate that *Traffic-MoE* (Ours) achieves performance highly comparable to “MoE→Dense” across all tasks. Notably, the MoE architecture achieves this level of performance while sparsely activating only a subset of expert parameters during inference, demonstrating superior **parameter efficiency**. Furthermore, in highly obfuscated and complex scenarios such as *ISCXTor2016 (Tor)* and *ISCXVPN2016 (Mixed)*, *Traffic-MoE* exhibits a distinct performance

advantage, surpassing the Dense architecture by 3.93% and 1.69% in Macro-F1 scores, respectively. This suggests that the “expert specialization” mechanism of the MoE architecture plays a pivotal role in handling heterogeneous traffic with severe feature overlap. Unlike shared-parameter Dense architectures, *Traffic-MoE* utilizes dynamic routing to direct mixed traffic to specific expert networks, thereby enabling the precise decoupling of complex traffic behaviors.

E.3.2 Necessity of Pre-training Strategy. To investigate the contribution of pre-training to the model’s representational capability, we remove the pre-training phase and train the model from scratch on downstream tasks, constructing the “w/o PT” ablation model. The experimental results strongly attest to the necessity of the “Pre-training and Fine-tuning” paradigm. Upon removing pre-training, performance across all architectures declines significantly. Specifically, the Macro-F1 score of *Traffic-MoE* (Ours) on the *ISCXVPN2016 (NonVPN)* dataset drops sharply by 23.14% (0.8005 → 0.6153). This confirms that universal traffic semantics acquired through large-scale unsupervised pre-training serve as a critical initialization foundation for achieving high performance in downstream tasks.

Furthermore, a profound observation is that the performance degradation of *Traffic-MoE* (Ours) without pre-training is generally more severe than that of the “MoE→Dense” ablation model (e.g., in the Tor scenario, the MoE architecture experienced a 21.80% drop

compared to 15.49% for the Dense architecture). This highlights the MoE architecture's acute dependency on prior knowledge: its efficacy hinges on the Router's decision-making capability. Without the robust feature space established by pre-training, a randomly initialized Router struggles to derive effective expert allocation strategies from limited labeled data, thereby preventing the model from converging to an optimal solution.

E.3.3 Role of Load Balancing Loss. To verify the role of the Auxiliary Loss $\mathcal{L}_{aux}$ in stabilizing MoE training, we evaluate an ablation model trained without this loss term (w/o $\mathcal{L}_{aux}$). Experimental results show that after removing the load balancing constraint, the model's Macro-F1 scores generally dropped by 0.06% to 2.70% across various tasks. This result validates the critical role of auxiliary loss in preventing "expert collapse". Without the regularization constraint of $\mathcal{L}_{aux}$, the gating network tends to fall into a local optimum, routing the majority of traffic tokens to a few dominant experts. Consequently, the remaining experts are not sufficiently updated, causing the model to degenerate into a capacity-limited dense model. The auxiliary loss enforces a uniform distribution of computational load, maximizing the utilization of the model's overall capacity.

E.3.4 Complementarity of Input Representations. By default, we construct flow sequences using "Header Features + Partial Payload". To deconstruct the contributions of each, we compare ablation models using only header features (w/ Header Only) and only payload (w/ Payload Only). Results indicate that header features generally play a dominant role in traffic detection. Particularly in the attack detection scenarios constructed from *CICIoMT2024* and *CICIoT2023*, the performance advantage of "w/ Header Only" ablation model is most significant (e.g., in *CICIoMT2024*, the Macro-F1 score leads "w/ Payload Only" ablation model by over 100%). This is because network attacks (e.g., DDoS, Brute Force) inherently manifest as anomalous traffic statistical behaviors (e.g., specific packet length sequences and inter-arrival patterns), and this side-channel information is fully preserved in packet headers. Conversely, attack payloads often consist of random padding or repetitive instructions, lacking discriminative semantic information, which leads to poor performance of "w/ Payload Only" ablation model in such tasks.

However, we observe a notable exception in the Tor scenario: "w/ Payload Only" ablation model (Macro-F1: 0.6996) actually outperformed "w/ Header Only" ablation model (Macro-F1: 0.6492). This counter-intuitive phenomenon is attributed to the Tor protocol's unique "fixed-size cell padding" mechanism. Tor enforces packet shaping into uniform 512-byte cells. This "traffic shaping" causes severe homogenization of the key "packet length sequence" information in header features, thereby drastically weakening their discriminative power. Under this specific constraint, the underlying application-level interaction patterns implicit in the payload sequence become a relatively more effective information source.

Despite the inherent limitations of individual modalities in specific scenarios (e.g., header obfuscation in Tor and payload semantic sparsity in attack detection), the complete model *Traffic-MoE* (Ours) consistently maintains superior performance. For instance, in the Tor scenario where header features are distorted, *Traffic-MoE* improves the Macro-F1 score to 0.8072 by fusing both feature types,

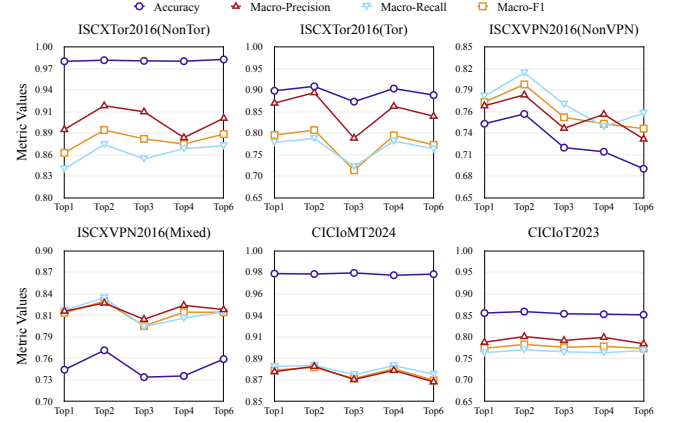


Figure 11: Impact of Top-k parameters on model performance.

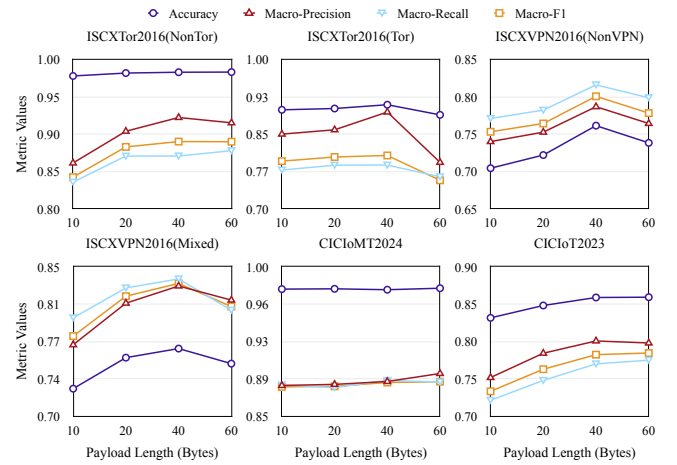


Figure 12: Impact of payload length on model performance.

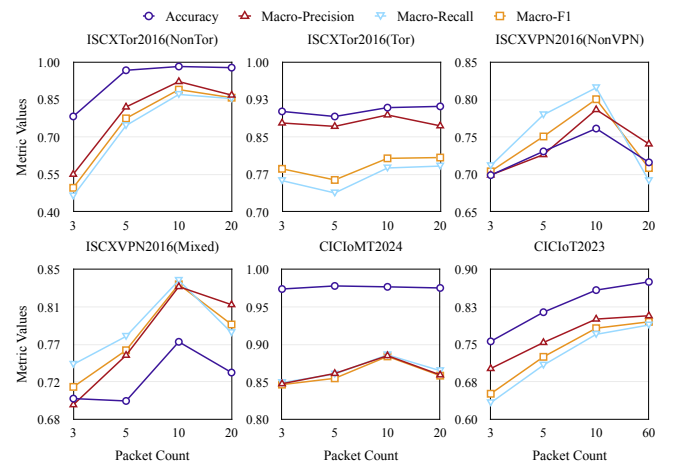


Figure 13: Impact of packet count on model performance.

Table 6: Performance comparison on datasets with different distribution shifts (OOD Generalization)

Method	CICIoMT2024 (Time-shift)				CICIoMT2024 (Proportion-shift)				CICIoMT2024 (Compose-shift)			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.5452	<u>0.4822</u>	<u>0.5203</u>	<u>0.4403</u>	0.8979	0.7697	<u>0.8859</u>	0.8057	0.7352	0.7234	<u>0.7846</u>	<u>0.7462</u>
FlowPrint	0.1772	0.2867	0.2689	0.1788	0.2328	0.2752	0.4250	0.2899	0.4365	0.3623	0.4256	0.2925
FS-Net	0.4584	0.3415	0.4262	0.3347	0.8579	0.7088	0.7937	0.7382	<u>0.7920</u>	0.7617	0.7518	0.7399
ET-BERT	<u>0.6507</u>	0.3755	0.3716	0.3382	<u>0.9298</u>	0.6747	0.7837	0.6984	<u>0.7367</u>	0.5988	0.7838	0.6435
NetGPT	0.4942	0.3588	0.4065	0.3458	0.8769	<u>0.7914</u>	0.8581	<u>0.8073</u>	0.7116	0.6735	0.7643	0.7108
TrafficFormer	0.5291	0.3915	0.4528	0.3513	0.8628	0.7735	0.8312	0.7808	0.7184	0.6419	0.7542	0.6167
Traffic-MoE	0.8219	0.7325	0.7209	0.7053	0.9727	0.8611	0.9366	0.8911	0.8071	<u>0.7301</u>	0.8585	0.7710

Method	CICIoT2023 (Time-shift)				CICIoT2023 (Proportion-shift)				CICIoT2023 (Compose-shift)			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.2217	0.3118	0.2121	0.2149	0.7232	0.6974	0.6536	0.6443	0.5831	0.6138	0.5759	0.5685
FlowPrint	0.6602	0.2835	0.2388	0.2199	0.3671	0.3817	0.4271	0.3174	<u>0.5907</u>	<u>0.5897</u>	0.4445	0.4297
FS-Net	0.2045	0.2480	0.1963	0.1892	0.6937	0.6562	0.6357	0.5993	0.5750	0.5714	0.5643	0.5515
ET-BERT	0.2987	0.2684	0.2433	0.2262	<u>0.7834</u>	<u>0.7267</u>	<u>0.7683</u>	<u>0.7190</u>	0.5637	0.5171	0.5692	0.5258
NetGPT	0.3854	<u>0.3279</u>	0.3145	0.2964	0.7535	0.7193	0.7398	0.7037	0.5508	0.5190	<u>0.5848</u>	0.5136
TrafficFormer	0.3993	0.3250	<u>0.3354</u>	<u>0.3115</u>	0.7559	0.7259	0.7286	0.7029	0.5597	0.5280	0.5813	0.5341
Traffic-MoE	<u>0.5031</u>	0.4723	0.4343	0.4316	0.8052	0.7280	0.7722	0.7281	0.6331	0.6222	0.6435	0.6123

representing a 15.38% improvement over “w/ Payload Only” ablation model. This result fully demonstrates the complementarity of header features and data payloads in the information space. Specifically, the model dynamically extracts features from the more reliable source and compensates for single-view defects through multi-view decision fusion, thereby achieving robust classification accuracy across heterogeneous and adversarial network environments.

E.4 Impact of Top- k Setup on Expert Activation

To investigate the optimal trade-off between model capacity and computational sparsity, we evaluate the impact of varying the number of active experts $k \in \{1, 2, 3, 4, 6\}$ on model performance, as illustrated in Figure 11. Overall, the performance trajectory exhibits a peak at $k = 2$, indicating that this configuration represents the Pareto-optimal point balancing accuracy and efficiency. However, an in-depth cross-task comparison reveals two distinct performance patterns, underscoring the differentiated demands imposed by varying traffic types on expert synergy.

In scenarios involving high obfuscation and encrypted traffic (e.g., *ISCXTor2016*, *ISCXVPN2016*), model performance demonstrates significant sensitivity. Specifically, shifting from $k = 1$ to $k = 2$ yields a substantial performance leap, confirming that resolving feature ambiguity in encrypted traffic necessitates the collaborative synergy of a “primary-auxiliary” expert pair. However, as k further increases, performance paradoxically degrades. We attribute this to noise injection: when processing such complex yet subtle features, forcing the activation of excessive experts introduces irrelevant “noise experts”. These low-confidence signals dilute the discriminative power of the dominant experts and may lead the model to overfit to spurious correlations in the training data.

Conversely, in attack detection tasks (e.g., *CICIoMT2024*), the model exhibits low sensitivity to varying k , where sparse settings ($k = 2$) achieve competitive or superior performance compared to the denser setting ($k = 6$). This phenomenon serves as compelling evidence for the validity of sparse activation. For attack traffic characterized by distinct and deterministic signatures (e.g., specific packet header sequences in brute-force attacks), a few specialized experts are sufficient to precisely capture these features. In this context, increasing k provides no marginal gain while incurring unnecessary computational overhead. Therefore, the default $k = 2$ setting not only provides necessary expert synergy for complex tasks but also replicates the performance ceiling of dense architectures in simpler tasks with minimal computational overhead.

E.5 Parameter Analysis on Traffic Sequence

E.5.1 Impact of Payload Truncation Length. To explore the effective semantic boundary within a single packet, we evaluate the impact of varying the payload truncation length J on model performance. In Figure 12, as J increases from 10 to 40 bytes, model performance rises rapidly to reach a peak. However, as J further exceeds 40 bytes, performance gains plateau and, in certain encrypted traffic tasks, even exhibit degradation. This phenomenon underscores the header semantic enrichment inherent in network traffic, where critical detection fingerprints (i.e., TLS record headers, cipher suite identifiers, and handshake lengths) are densely clustered within the packet’s initial segment. Conversely, trailing payloads beyond this threshold predominantly consist of high-entropy encrypted data or random padding. These regions lack discriminative semantics and function as stochastic noise that impairs the attention mechanism’s ability to focus on salient features. Consequently, we calibrate J to 40 bytes. This window effectively encompasses the critical headers

of the vast majority of protocols, thereby preserving high-density discriminative information while minimizing token sequence length to reduce inference FLOPs.

E.5.2 Impact of Packet Count. Regarding the number of packets K in the traffic sequence, we observe a similar law of diminishing marginal utility, as illustrated in Figure 13. Experiments indicate that model performance achieves substantial growth in the interval from $K = 3$ to $K = 10$ but stagnates or even declines for $K > 10$. This trend aligns with the fundamental interaction mechanism of network protocols, where the primary discriminative signals are encoded in the “protocol grammar” during the initial connection establishment (e.g., TCP three-way handshakes, TLS key exchanges, and certificate negotiations). Interaction patterns in this early phase are highly deterministic and class-specific. Once the connection enters the subsequent bulk transfer phase, traffic behavior becomes homogenized (characterized mainly by continuous payload delivery), providing minimal incremental information. Consequently, selecting the first $K = 10$ packets as the input window is sufficient to capture the complete session establishment logic while effectively avoiding the quadratic growth in computational complexity associated with processing long sequences, thus achieving efficient representation for long flows.

E.6 Evaluation under Distribution Shifts

Real-world network environments are highly dynamic, characterized by continuous temporal evolution and non-stationary class distributions. To rigorously evaluate whether *Traffic-MoE* can adapt to such “open-world” challenges without frequent retraining, we construct three out-of-distribution (OOD) benchmarks based on the *CICIoMT2024* and *CICIoT2023* datasets: **Time-shift**, **Proportion-shift**, and **Compose-shift**.

- **Time-shift** is designed to simulate concept drift. For fine-grained traffic classification tasks (e.g., distinguishing between DDoS-SYN-Flood, DDoS-SlowLoris, and DDoS-ACK-Fragmentation, which all belong to the DDoS category), we strictly sort all session flows of each category by timestamp. We designate the first 40% of the entire time span as the training domain and the last 40% as the testing domain, while discarding the middle 20% to establish a temporal buffer. This “past predicting future” setting eliminates temporal data leakage and ensures a significant temporal distributional gap.
- **Proportion-shift** evaluates the model’s adaptability to intra-class distribution shifts. For coarse-grained traffic classification tasks (e.g., detecting attacks where distinct variants such as DDoS-SlowLoris and DDoS-SYN-Flood are all classified as the coarse-grained DDoS category), we artificially adjust the ratio of dominant to minor components in each coarse-grained category (e.g., DDoS, WebAttack): the training set is sampled with a 4:1 ratio, whereas the testing set adopts a completely inverse 1:4 ratio. This extreme distribution inversion rigorously tests the model’s capacity to capture long-tail features.
- **Compose-shift** simulates unseen attack sub-variants. In coarse-grained traffic classification tasks, we randomly mask 50% of fine-grained sub-class traffic in each coarse-grained category during training, while the testing set includes all sub-classes, thereby

simulating the real-world challenge of “missing information” due to emerging attack variants.

Table 6 presents the performance comparison under three distinct distribution shift scenarios. The results consistently demonstrate that *Traffic-MoE* exhibits superior robustness and generalization compared to both statistical and dense deep learning baselines, compellingly validating the effectiveness of the sparse MoE architecture in handling complex environmental dynamics.

E.6.1 Resilience to Concept Drift (Time-shift). The Time-shift scenario represents the most rigorous test for model robustness. As shown in Table 6, *Traffic-MoE* achieves an overwhelming advantage, recording Macro-F1 scores of 0.7053 and 0.4316 on *CICIoMT2024* and *CICIoT2023* respectively, outperforming the runner-up by **60.18%** and **38.56%**. This disparity reveals a fundamental limitation of dense baselines (e.g., ET-BERT), which tend to rely on *temporal shortcuts*—transient statistical artifacts specific to the training time-frame (e.g., specific burst timings)—rather than intrinsic attack behaviors. When the temporal context shifts, these shortcuts become invalid. In contrast, *Traffic-MoE* mitigates this through **sparse expert specialization**. By leveraging pre-trained protocol knowledge and selectively activating experts, it effectively disentangles time-invariant protocol semantics from transient noise, ensuring robust detection even when concept drift occurs.

E.6.2 Stability under Structural Shifts (Proportion & Compose-shift). In scenarios involving structural mutations in traffic composition, *Traffic-MoE* demonstrates exceptional generalization capabilities, maintaining SOTA performance in attack detection.

1) Mitigation of Intra-class Distribution Shift (Proportion-shift): *Traffic-MoE* continues to outperform all competitors, suggesting that the MoE architecture effectively mitigates the *gradient starvation* of minority classes typically encountered in dense networks. The “divide-and-conquer” dynamic routing allows long-tail minor components to be handled by specific experts, preventing their gradient updates from being diluted by dominant classes, thus preserving high recall even under inverse test distributions.

2) Adaptation to Unseen Variants (Compose-shift): *Traffic-MoE* ranks first in core metrics on both datasets. Notably, while FS-Net achieved higher Macro-Precision in *CICIoMT2024*, *Traffic-MoE* surpasses it by a significant margin of **14.19%** in Macro-Recall. High recall on unseen sub-classes indicates that *Traffic-MoE* has moved beyond simple *signature memorization* to achieve robust behavioral abstraction (e.g., recognizing the generic pattern of “flooding” rather than tool-specific payloads). This capability is critical for defending against emerging threats, where evasion (False Negative) is far more costly than a false alarm.

In summary, these results validate *Traffic-MoE* not merely as a high-performing classifier, but as a resilient foundation model for operational network defense. By effectively handling the inherent *non-stationarity* of real-world network traffic, *Traffic-MoE* maintains continuous situational awareness without the severe performance degradation often observed in dense models. Crucially, this superior generalization and robustness enable sustained adaptation to long-term evolving network environments, significantly reducing the frequency and overhead of model retraining in real-world deployments.